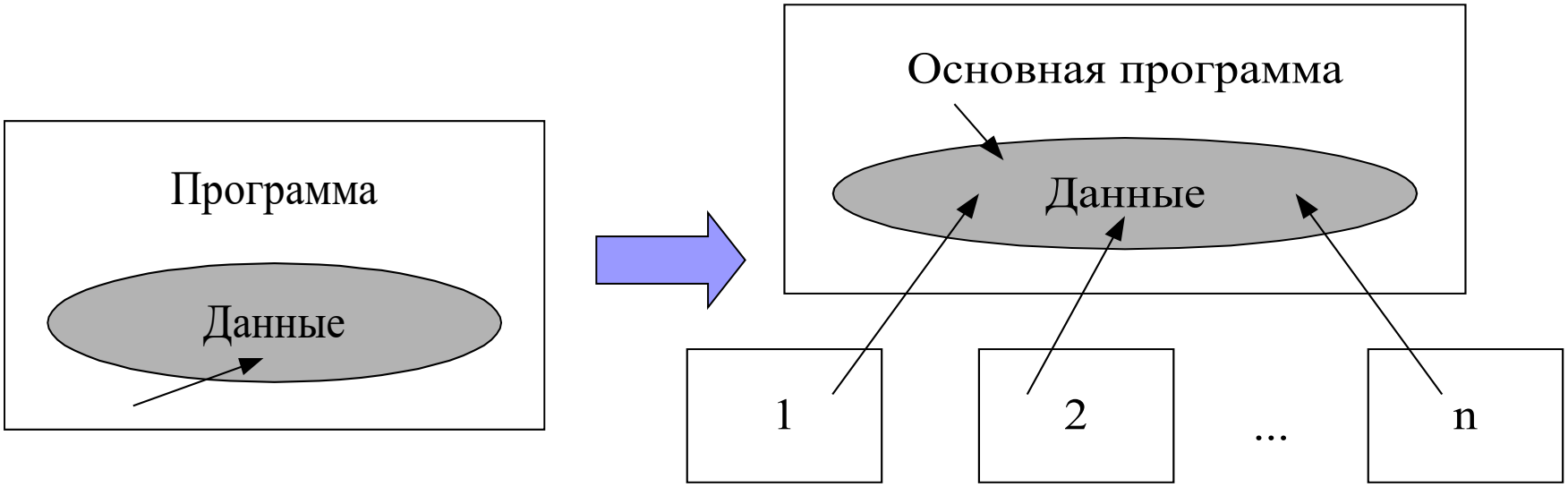
2024

# Глава 7 Основы объектно-ориентированного программирования

МГТУ им. Н.Э. Баумана
Факультет Информатика и системы управления
Кафедра Компьютерные системы и сети
Лектор: д.т.н., проф.
Иванова Галина Сергеевна

# Введение. Эволюция технологии разработки ПО. Процедурная и объектная декомпозиция

1. «Стихийное» программирование – до середины 60-х годов XX века – технология отсутствует – программирование – искусство создания программ – в конце периода появляется возможность создания подпрограмм – используется процедурная декомпозиция.

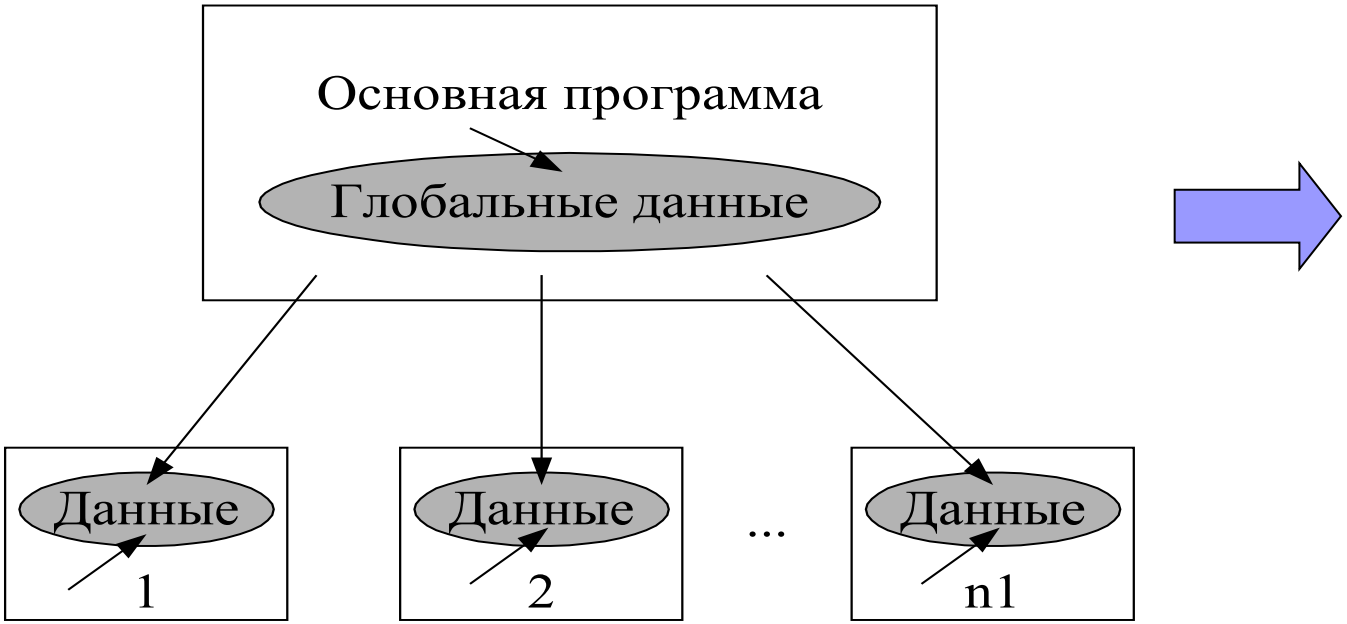



Слабое место – большая вероятность испортить глобальные данные.

# Эволюция технологии разработки ПО (2)

2. Структурный подход к программированию - 60-70-е годы XX века – технология, представляющая собой набор рекомендаций и методов, базирующихся на большом опыте работы:

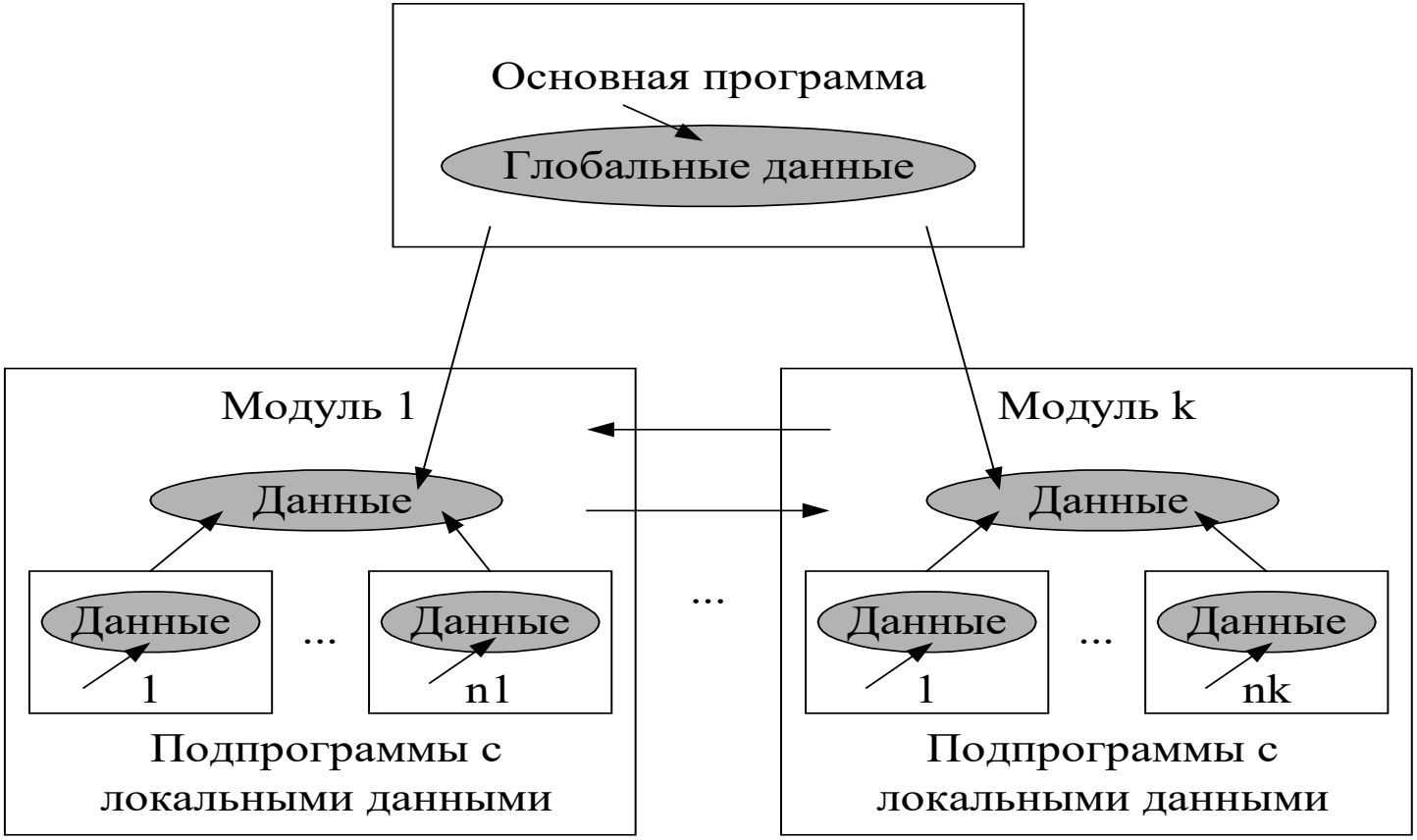
- нисходящая разработка;
- декомпозиция методом пошаговой детализации;
- структурное программирование;
- сквозной структурный контроль и т. д.




Подпрограммы с локальными данными

# Эволюция технологии разработки ПО (3)

**Модульное программирование** – выделение групп подпрограмм, использующих общие глобальные данные в модули – отдельно компилируемые части программы (многоуровневая декомпозиция).



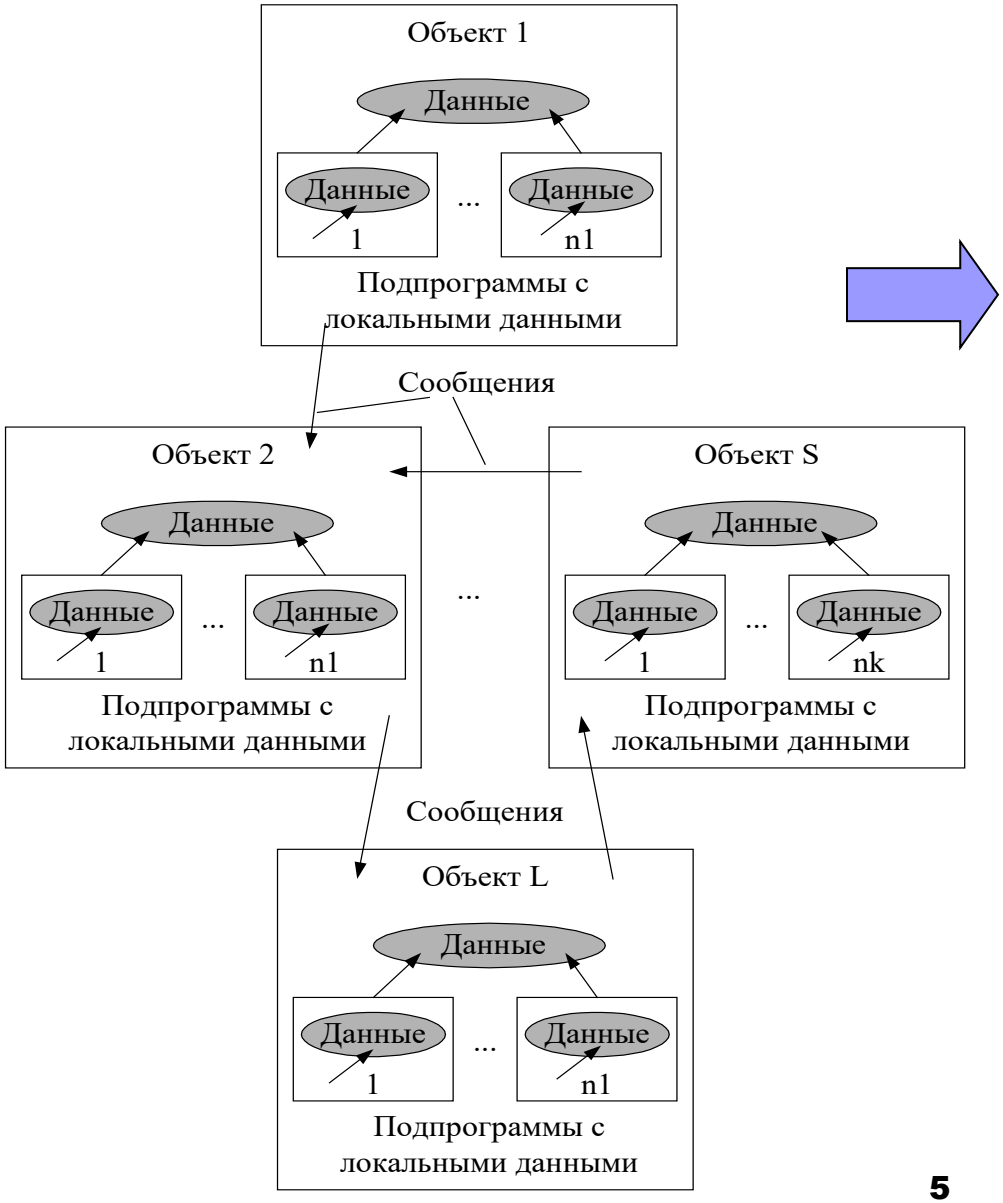

Слабое место – большое количество передаваемых параметров.

# Эволюция технологии разработки ПО (4)

**3. Объектный подход к программированию –** с середины 80-х до наших дней**.**

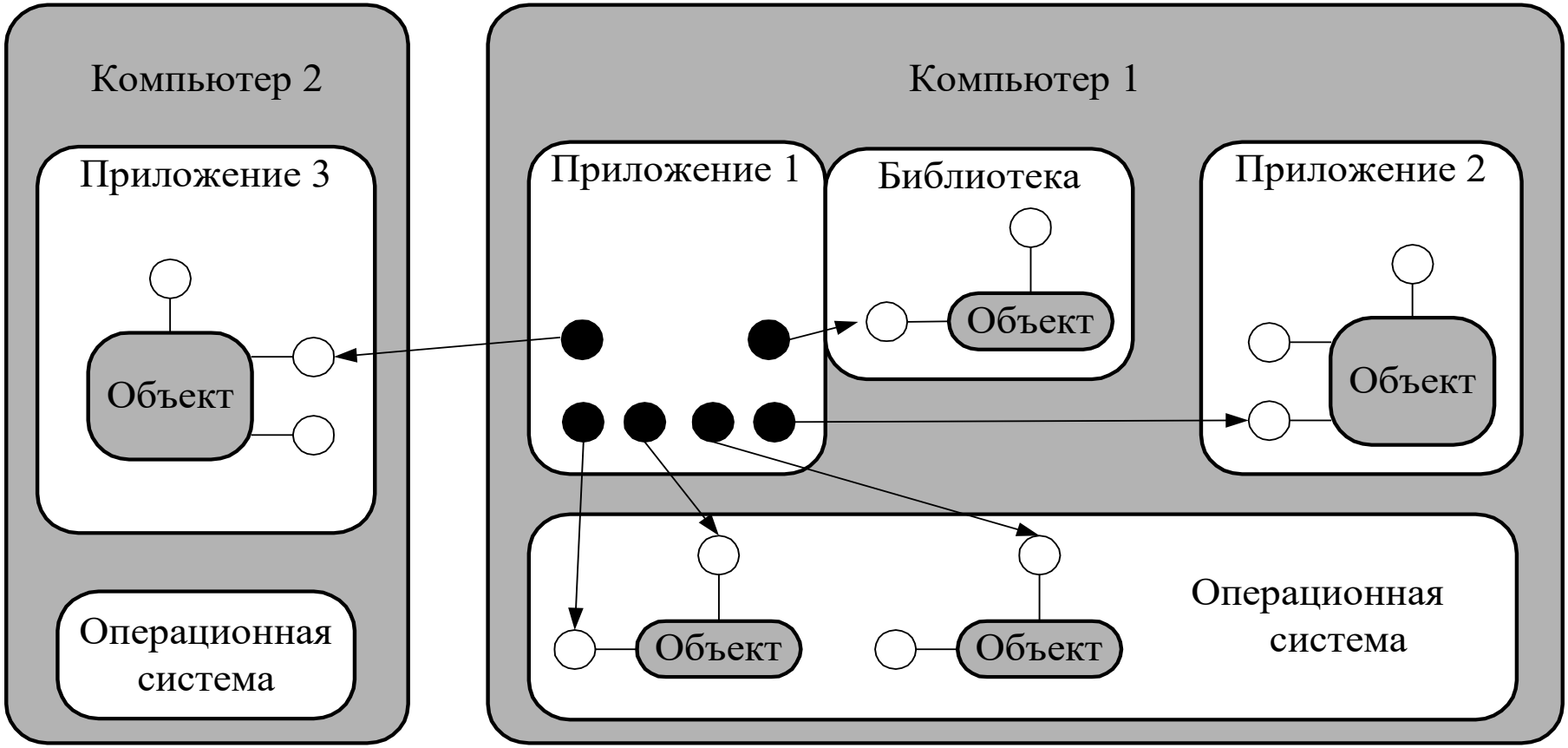
**Объектно-ориентированное программирование** – технология создания сложного программного обеспечения, основанная на представлении программы в виде системы объектов, каждый из которых является экземпляром определенного типа (класса),

а классы образуют иерархию с наследованием свойств.



# Эволюция технологии разработки ПО (5)

**Компонентный подход** – с конца 90-х годов XX века (COM-технология, Corba, SOAP) – подключение объектов через универсальные интерфейсы – развитие сетевого программирования – появление CASE-технологий.



# Пример

Разработать программную систему, которая для указанной функции на заданном отрезке:

- строит таблицу значений с определенным шагом;
- определяет корни;
- определяет максимум и минимум.

# Формы интерфейса пользователя

Программа исследования функций.
Введите функцию или слово «Конец»: y = cos(x) – 1
Назначьте интервал: [-1, 0)

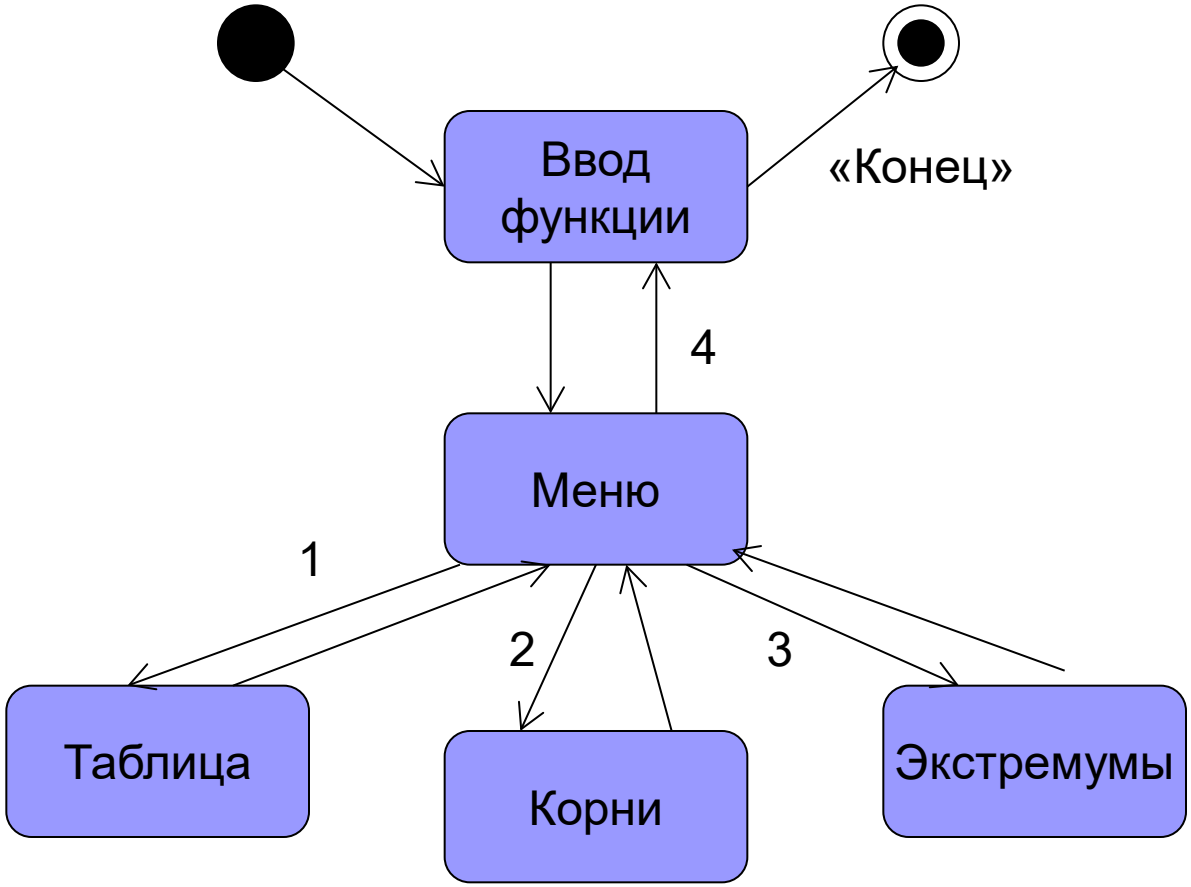
Введите номер решаемой задачи
( 1 – построение таблицы значений;
2 – нахождение корней;
3 – нахождение минимума и максимума;
4 – смена функции или завершение программы): 1

Построение таблицы.
Введите шаг: 0.01
Таблица значений:
x= y=
…

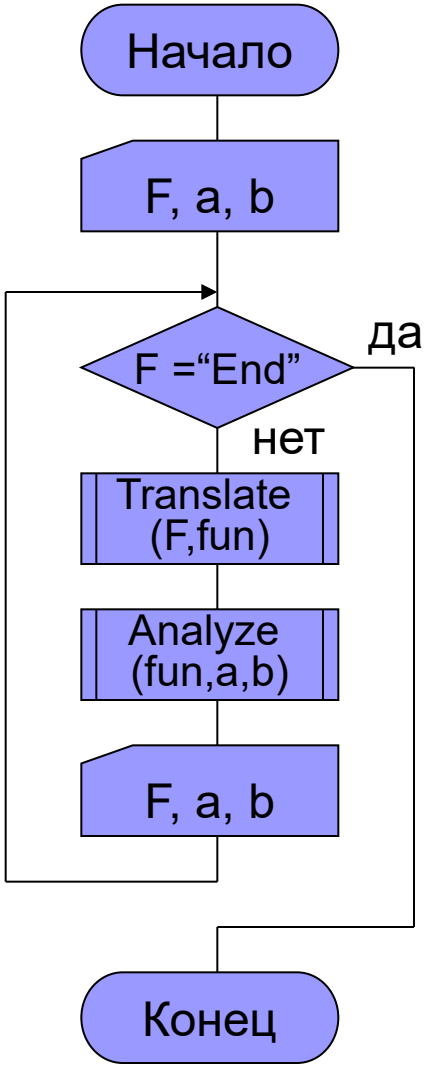
Нахождение корней.
Таблица корней:
x= y=
…

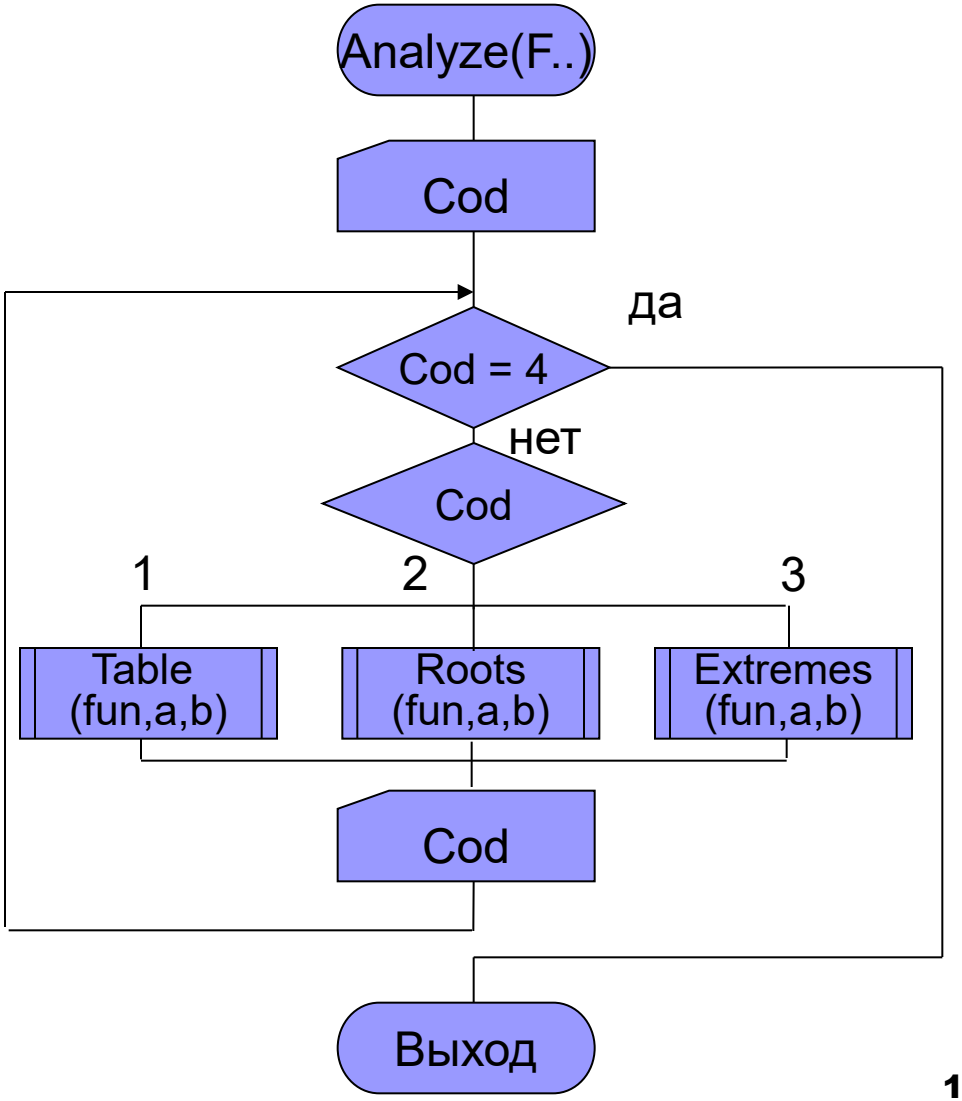
Экстремумы.
Минимум:
x= y=
Максимум:
x= y=

# Диаграмма состояний интерфейса пользователя

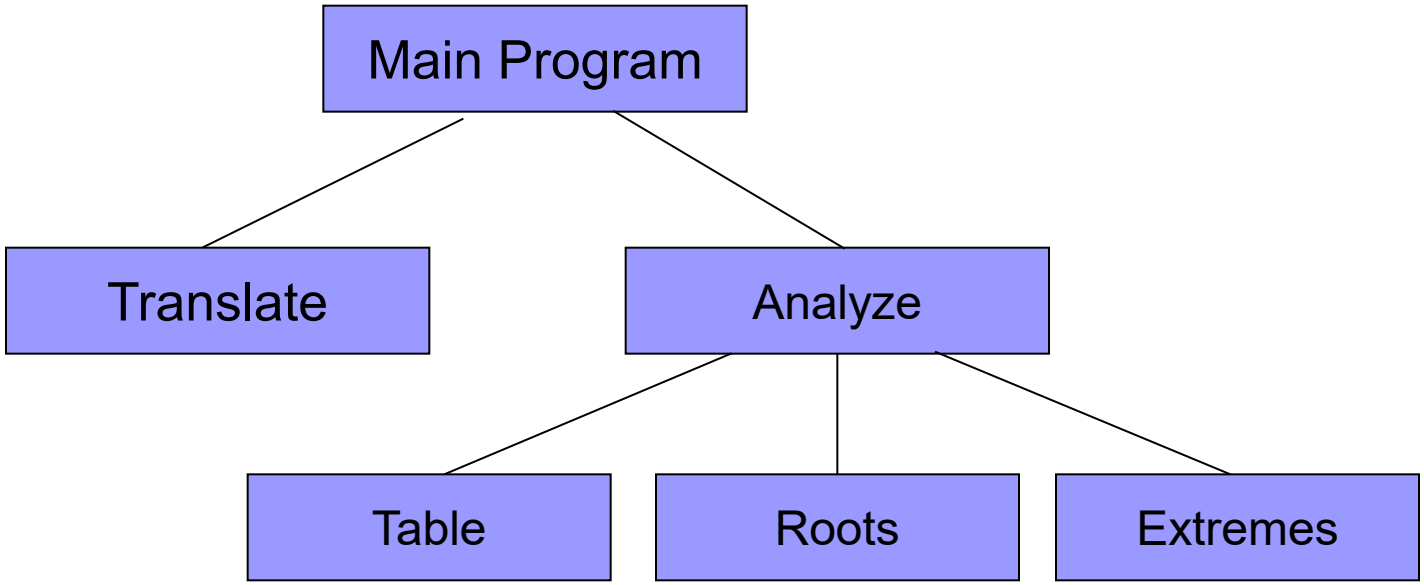


# Разработка схем алгоритмов методом пошаговой детализации



# Схема структурная программы




**Процедурная декомпозиция –** процесс разбиения программы на подпрограммы.

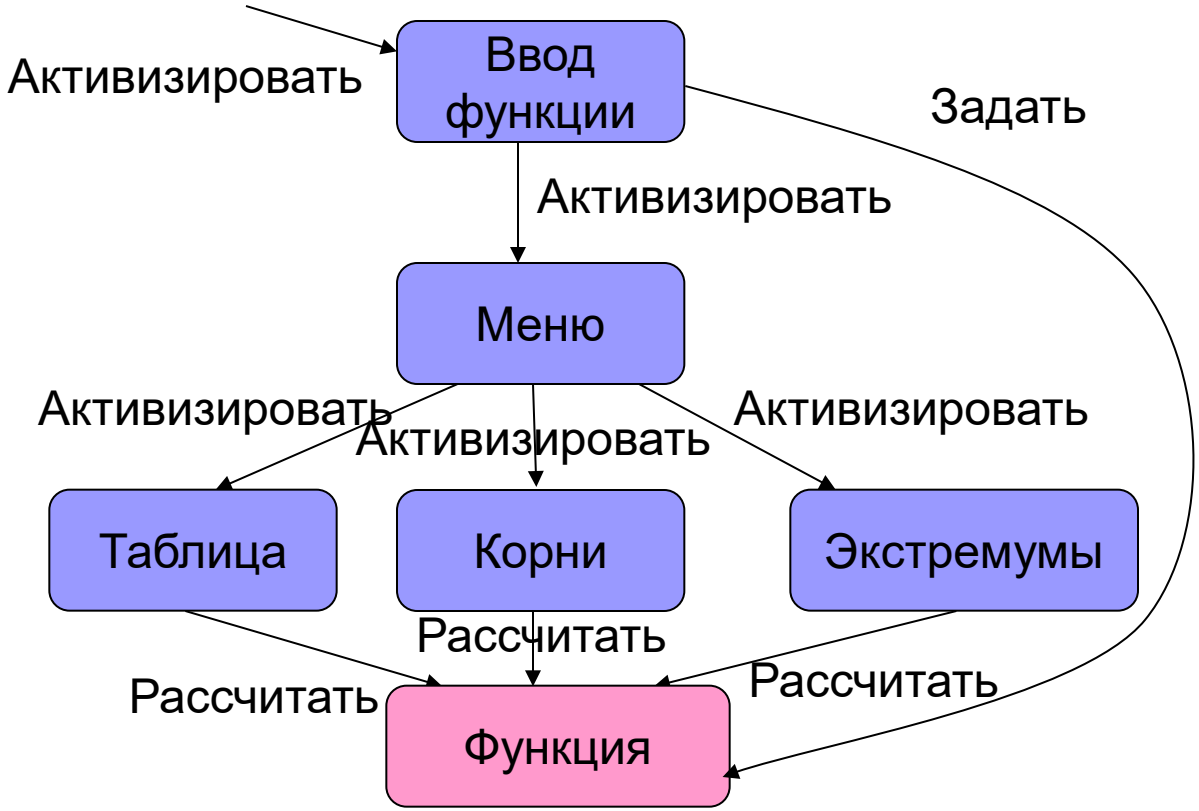
**Структурной** называют декомпозицию, если:

- каждая подпрограмма имеет один вход и один выход;
- подпрограммы нижних уровней не вызывают подпрограмм верхних уровней;
- размер подпрограммы не превышает 40-50 операторов;
- в алгоритме использованы только структурные конструкции.

# Объектная декомпозиция

**Объектная декомпозиция** – процесс представления предметной области задачи в виде отдельных функциональных элементов (объектов предметной области), обменивающихся в процессе выполнения программы входными воздействиями (сообщениями) .

Объект отвечает за выполнение некоторых действий, инициируемых сообщениями и зависящих от параметров объекта.



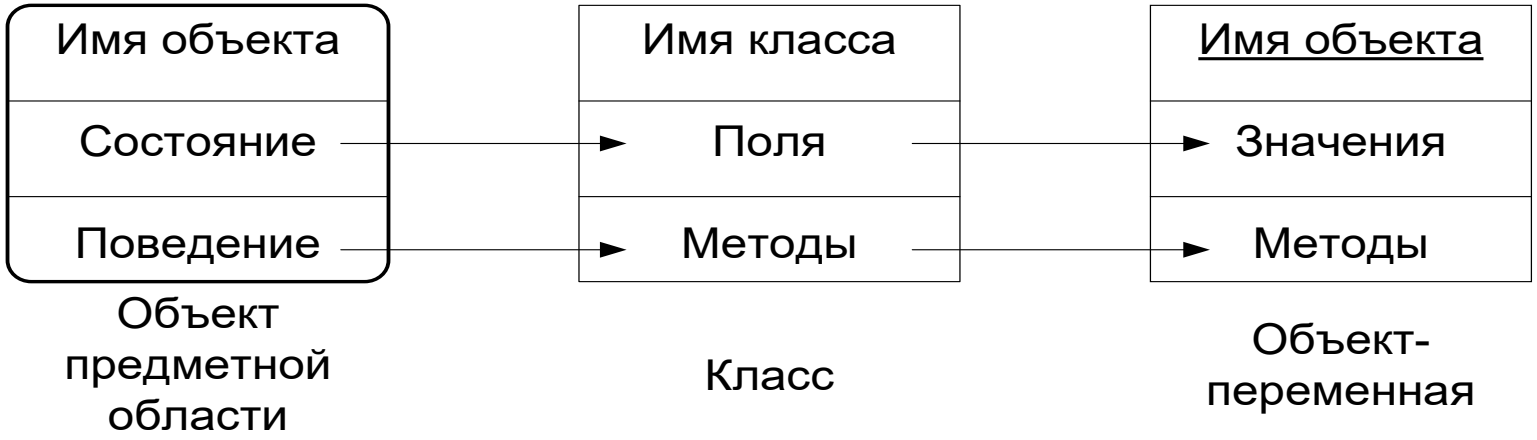

**Объект предметной области** характеризуется:

- именем;
- состоянием;
- поведением.

**Состояние** – совокупность значений характеристик объекта, существенных с т. з. решаемой задачи.

**Поведение** – совокупность реакций на сообщения.

# Реализация объектов предметной области




***Класс*** – это структурный тип данных, который включает описание полей данных, а также процедур и функций, работающих с этими полями данных.

Применительно к классам такие процедуры и функции получили название ***методов***.

***Объект-переменная*** – переменная типа «класс».

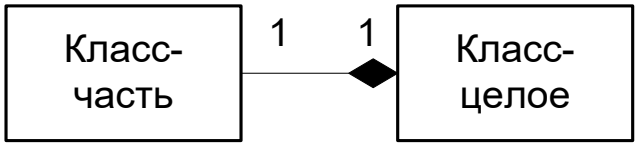
# Основные принципы, на которых базируется объектно-ориентированное программирование

- ***абстрагирование*** – выделение абстракций в предметной области задачи; под ***абстракцией*** при этом понимается совокупность существенных характеристик некоторого объекта предметной области, которые отличают его от всех других видов объектов;
- ***инкапсуляция*** – размещение в одном программном компоненте (объекте) данных и подпрограмм, которые с этими данными работают;
- ***ограничение доступа*** – сокрытие отдельных элементов реализации абстракции, не затрагивающих существенных характеристик ее как целого;
- ***модульность*** – принцип разработки программной системы, предполагающий реализацию ее в виде отдельных частей;
- ***иерархичность*** – использование иерархий при разработке программных систем; при этом используются как иерархии "целое-часть", так и иерархии "общее-частное";
- ***типизация*** – ограничение, накладываемое на свойства объектов и препятствующее взаимозаменяемости абстракций различных типов (или сильно сужающее возможность такой замены).

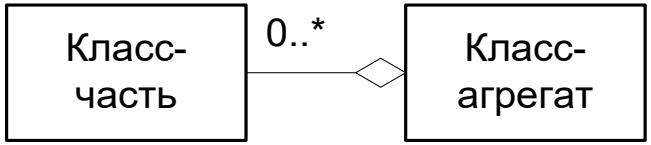
# Методы построения классов

1. ***Наследование*** – *механизм*, позволяющий строить класс на базе более простого посредством добавления полей и определения новых методов.
   При этом исходный класс, на базе которого выполняется построение, называют *родительским* или *базовым*, а строящейся класс – *потомком* или *производным* классом.
   Если при наследовании какие-либо методы переопределяются, то такое наследование называется ***полиморфным***.

2. ***Композиция*** – *механизм*, позволяющий включать один или несколько объектов других классов в объекты конструируемого.




3. ***Наполнение*** – *механизм*, позволяющих включать или не включать объекты других классов в объект конструируемого..



2024

# Глава 7 Средства объектно-ориентированного программирования

МГТУ им. Н.Э. Баумана

Факультет Информатика и системы управления

Кафедра Компьютерные системы и сети

Лектор: д.т.н., проф.

Иванова Галина Сергеевна

# 7.1 Определение класса, инкапсуляция полей и методов класса. Объявление объектов и инициализация полей

С точки зрения синтаксиса ***класс*** – структурный тип данных, в котором помимо полей разрешается описывать *прототипы* (заголовки) процедур и функций, работающих с этими полями данных.

```
class Имя_класса {
    [private:]   Внутренние_компоненты_класса;
    protected:   Защищенные_компоненты_класса;
    public:      Общедоступные_компоненты_класса;
};
```

По умолчанию компоненты считаются private!

***Внутренние*** компоненты класса – поля и методы доступны только методам класса.

***Защищенные*** – доступны методам своего класса и методам классов-наследников.

***Общедоступные*** – доступны в пределах видимости, в том числе из программы и методов других классов.

# Пример. Площадь комнаты

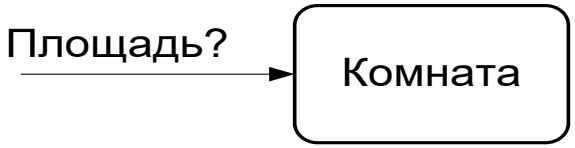



Диаграмма объектов

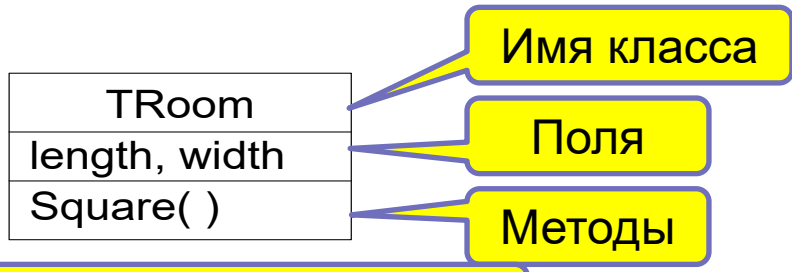



Диаграмма классов (UML)

1. При объявлении класса *строится его пространство имен*.
2. Тело метода можно описать:
   а) внутри объявления класса, тогда оно по умолчанию **`inline`**;
   б) после объявления класса в пространстве его имен.
3. Поскольку данные и методы ***инкапсулированы*** в пространстве имен класса, все поля *автоматически доступны* из любого метода класса.

```
class TRoom {
   private: float length, width;
   public:  float Square(); // прототип метода
};

float TRoom::Square() { // тело метода в пространстве имен класса
  return length*width;
}
```

# Связь между полями и методами класса. Неявный параметр this

Любой метод *неявно* получает параметр **this** – указатель на объект (адрес полей объекта), и обращение к полям и методам объекта выполняется с использованием этого адреса.

```
float TRoom::Square()
{
    return this -> length * this -> width;
}
```

Этот параметр при обращении к полям и методам класса *можно не указывать*, так как он подразумевается по умолчанию:

```
float TRoom::Square()
{
    return length * width;
}
```

# Объявление объектов класса

**Примеры:**

```
TRoom A,        // объект A класса TRoom
      B[5],     // массив объектов класса TRoom
      *pC;      // указатель на объект класса TRoom
```

Для создания динамического объекта необходимо выделить память:

```
pC = new TRoom;
```

а после его использования – освободить память:

```
delete pC;
```

Доступ к полям и методам аналогичен доступу к полям структуры:

**Примеры:**

а) `s = A.Square();`

б) `s += B[i].Square();`

в) `s = pC -> Square();`

Из программы можно обращаться только к открытым (public) компонентам класса!

# Инициализация полей прямой записью в поле

Если поля объявлены в области public, то их можно инициализировать в тексте программы:

Ex07_01a

```
#include <iostream>
using namespace std;
class TRoom {
   public: float length, width;
           float Square()
           {
               return length* width;
           }
};

int main(){
    TRoom A;
    A.length = 3.5;
    A.width = 5.1;
    cout << "S = " << A.Square() << endl;
}
```

Из программы можно обращаться только к открытым компонентам класса!

# Инициализация полей при объявлении объекта

Если поля объявлены в области public, то их можно инициализировать при создании объекта, как поля структуры:

Ex07_01b

```
#include <iostream>
using namespace std;

class TRoom {
   public: float length, width;
           float Square();
};
float TRoom::Square(){
   return length* width;
}

int main(){
    TRoom A = {3.5,5.1}; // операция "=" не обязательна
    cout << "S = " << A.Square() << endl;
}
```

Из программы можно обращаться только к открытым компонентам класса!

# Инициализация посредством инициал. метода

Скрытые поля можно инициализировать только используя метод класса:

Ex07_01c

```
#include <iostream>
using namespace std;
class TRoom {
   private: float length, width;
   public:  float Square();
            void Init(float l, float w);
};
void TRoom::Init(float l, float w){
   length = l;
   width = w;
}
float TRoom::Square(){
   return length* width;
}
int main(){
    TRoom A;
    A.Init(3.5,5.1);
    cout << "S = " << A.Square() << endl;
}
```

Метод посредством this может обращаться ко всем компонентам класса!

# 7.2 Конструкторы и деструкторы

***Конструктор*** – метод, автоматически вызываемый при выделении памяти под объект. Используется для инициализации полей объекта. Автоматический вызов страхует программиста от ошибки оставить поля неинициализированными.

```
class CBook
{ private:  char Name[30];
      int Pages;
  public:
      CBook(const char *name,int pages) {
          Pages=pages;
          strcpy(Name,name);
  }…
```

Имя конструктора совпадает с именем класса

При создании объектов классов с конструкторами параметры указывают в круглых скобках:

```
int main()
{  CBook D("J.London. Smoke Bellew",267);    …
```

# Конструкторы без параметров

Конструкторы, как и другие функции, можно перегружать. Специальный конструктор без параметров (инициализирующий или неинициализирующий) используется для создания объектов, которым при выделении памяти не могут быть переданы значения полей.

**а)** *неинициализирующий конструктор без параметров* используется для создания неинициализированных объектов, например:

```
CBook(){}
```

б) *инициализирующий конструктор без параметров* создает стандартно инициализированные объекты , например:

```
CBook(){ Name[0]='\0'; Pages=0; }
```

Класс может содержать единственный конструктор без параметров, либо инициализирующий, либо неинициализирующий.

При создании объектов посредством конструкторов без параметров *круглые скобки не указывают*:

```
void main()
{  CBook J; …
```

# Делегирующие конструкторы

Начиная с C++11, одни конструкторы класса (*делегирующие*) могут вызывать другие, объявленные в том же классе:

```
class CBook
{
    protected:    char Name[30];
       int Pages;
    public:
       CBook(const char *name,int pages) {
         Pages=pages;  strcpy(Name,name);
       }
       CBook():CBook("No name",0){}
 };
```

Это позволяет избежать дублирования кода и связанных с ним ошибок.

# Пример. Инициализация объекта конструктором

Файл Room.h:

Ex07_02

```
#ifndef ROOM_H
#define ROOM_H
class TRoom {
    private: float length, width;
    public:
        TRoom(float l, float w){
            length = l;
            width = w;
        }
        float Square(){
            return length* width;
        }
};
#endif // ROOM_H
```

Реализация методов описана в теле класса => все методы по умолчанию inline!

Файл main.cpp:

```
#include <iostream>
#include <Room.h>
using namespace std;

int main(){
    TRoom A(3.5,5.1);
    cout << "S = " << A.Square() << endl;
}
```

# Различные способы создания объектов

Ex07_03

```
class CPoint
{ private: int x,y;
  public: CPoint(int ax,int ay){x=ax;y=ay;}
          CPoint(){}
          void SetPoint(int ax,int ay) {x=ax; y=ay;}
          void Print(){...}…};
void main()
{
    CPoint A, B(2,3); // или B{2,3}; или B = {2,3};
    A.SetPoint(2,3); A.Print();
    B.Print();

    CPoint *E, *I = new CPoint(4,6);
    E = new CPoint(3,4); E->Print(); delete E;
    I->Print(); delete I;
}
```

"Простые" объекты

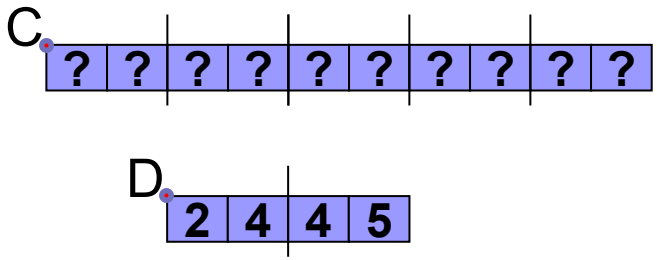
Динамические объекты

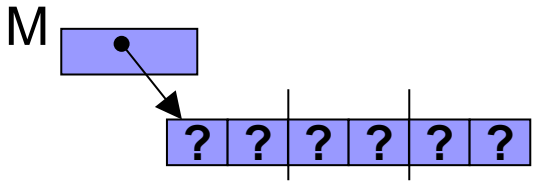
# Различные способы создания объектов (2)

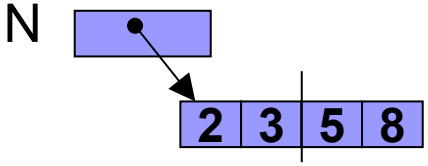
Массивы объектов

Динамические массивы объектов

```
void main()
{
    CPoint C[5],  D[2] = {CPoint(2,4),CPoint(4,5)};
        // или D[2]{{2,4},{4,5}};
    for (int i=0;i<5;i++) {C[i].SetPoint(i,i+1); C[i].Print();}
    for(int i=0;i<2;i++) D[i].Print();

    CPoint *M = new CPoint[5],
         *N = new CPoint[2]{CPoint(2,4),CPoint(4,5)};
            // или *N = new CPoint[2]{{2,4},{4,5}};
    for (i=0;i<3;i++){M[i].SetPoint(i,i+1); M[i].Print();}
    for (i=0;i<2;i++) N[i].Print();
    delete [] M; delete [] N;
}
```



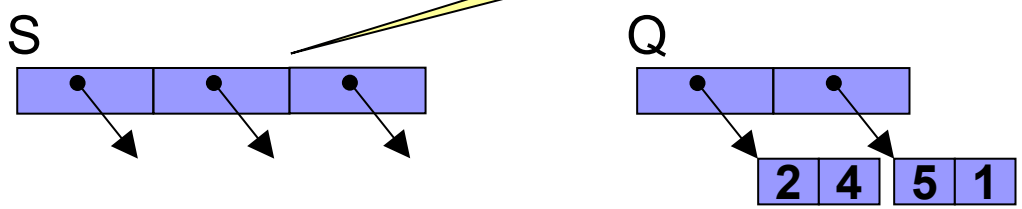



# Различные способы создания объектов (3)

```
void main()
{
    CPoint *S[3], *Q[] = {new CPoint(2,4),new CPoint(5,1)};
    for (int i=0;i<3;i++)
       {   S[i]=new CPoint(i,i+1);   S[i]->Print();   }
    for (int i=0;i<2;i++) Q[i]->Print();
    for (int i=0;i<3;i++) delete S[i];
    for (int i=0;i<2;i++) delete Q[i];
}
```

Массивы указателей

Конструктор не вызывается…



# Список инициализации. Инициализация константных, ссылочных и объектных полей

***Список инициализации*** – конструкция, используемая в конструкторе для вызова конструкторов полей и базового класса. Вне списка инициализации нельзя инициализировать поля базового класса, константны, ссылочные и объектные поля.

$ Элемент_списка_инициализации = **Имя_поля (Список_выражений)**

**Примеры:**

a)
```
TPoint(int ax,ay): x(ax),y(ay) {}
```

b)
```
class TLine
{ private:
      const int x;  // константные поля
      int &y;       // ссылочные поля
      TPoint t;     // объектные поля
  public: TLine(int ax,int ay,int tx,int ty):
                x(ax),y(ay),t(tx,ty){}
          TLine(){}
  ...};
```

Автоматически вызывает конструктор объектного поля без параметров TPoint( ), если такого в классе TPoint нет, то ошибка!

# Объекты с динамическими полями. Деструкторы

**Деструкторы** аналогично конструкторам вызываются автоматически, но в момент освобождения памяти, выделенной под объект. Деструкторы обычно используют для освобождения памяти, выделенной под динамические поля объектов, например

```
class CBook
{  char *pName;    int Pages;
public:
   CBook(const char *name,int pages){
        pName=new char[30];
        strcpy(pName,name);
        Pages=pages;
   }
   ~CBook(){
        delete [] pName;
   }
};
```

Имя деструктора совпадает с именем класса, но перед ним ставится символ тильда "~"

# Копирующий конструктор

Автоматически вызывается:

а) при использовании объявлений вида:

```
TPoint A(2,5),B = A;
```

б) при передаче параметров-объектов по значению, например:

```
void Print(TPoint R) {…}
```

Компилятор автоматически строит копирующий конструктор "по умолчанию", выполняющий копирование полем методом "поле за полем". Но может быть требуемый описан программистом.

Формат:

```
Имя_конструктора(const Имя_класса & Имя_объекта){…}
```

**Примеры:**

а)
```
TPoint(const TPoint &Obj)
   {x=Obj.x; y=Obj.y;}
```

**Строится автоматически**

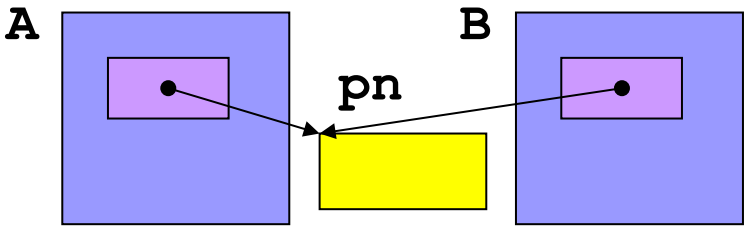
б)
```
TPoint(const TPoint &Obj)
   {x=Obj.x; y=2*Obj.y;}
```

**Пример объявленного в программе**

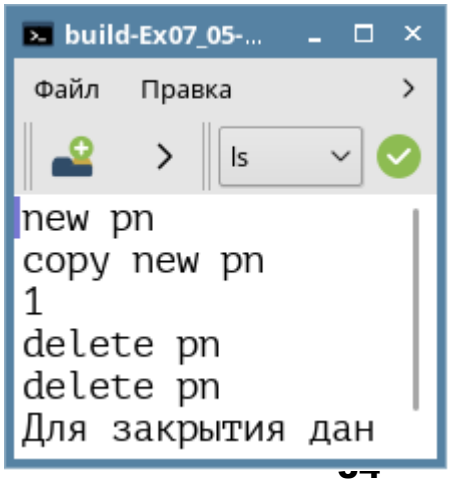
# Пример обязательного определения копирующего конструктора

Ex07_04

Копирующий конструктор обязателен для классов с динамическими полями!




```
#include <stdio.h>
class TNum
{ public:   int *pn;
      TNum(int n){puts("new pn"); pn=new int(n);}
      TNum(const TNum &Obj)
         {puts("copy new pn"); pn=new int(*Obj.pn);}
      ~TNum(){puts("delete pn");delete pn;}
};
void Print(TNum b) {
   printf("%d\n",*b.pn); }
int main() {
   TNum A(1);  Print(A);
   return 0;
}
```



# 7.3 Пример класса. Класс string

***Строка C++*** – объект класса **`string`**. Для работы со строкой необходимо подключить библиотеку: **`#include <string>`**

Конструкторы:

**`string()`** - создает пустую строку (по умолчанию);

**`string(string &S)`** - копия строки S;

**`string(size_t n, char c)`** - повторение символа c заданное число n раз;

**`string(size_t c)`** - строка из одного символа c;

**`string(string &S, size_t start, size_t len)`** - строка, содержащая не более, чем len символов данной строки S, начиная с символа номер start

**Примеры:**

```
string s1;                  // s1 – пустая строка
string str(10,'z');         // str – строка “zzzzzzzzzz”
string ss1("abc");          // ss1 – строка "abc“
```

# Класс string. Операции

Со строками можно выполнять следующие операции:

**=** - присваивание значения.

**+=** - добавление в конец строки другой строки или символа.

**+** - конкатенация двух строк, конкатенация строки и символа.

**==**, **!=** - посимвольное сравнение.

**<**, **>**, **<=**, **>=** - лексикографическое сравнение.

**Примеры**:

```
string S,S1("abc"),S2;
S2 = "abd";
S = S1 + S2;
S1 += S2;
if (S1 == S2)cout << "Строки равны." << endl;
```

# Класс string. Методы

Класс string содержит следующие методы:

**size()= length()** – определение длины строки;

**resize(int n[,char c])** – увеличение|уменьшение длины строки, если указан параметр c, то при увеличении длины добавляются символы c;

**clear()** – очистка строки;

**empty()** – проверка, пустая ли строка (возвращает true, если строка пустая);

**push_back(char c)** – добавление символа c в конец строки;

**append(…)** – добавление в конец строки: n заданных символов, другой строки или части другой строки;

**erase(…)** – удаление части строки с указанного номера до конца или с указанного номера заданного количества символов;

**insert(…)** – вставка, начиная с заданного символа в строку n заданных символов, другой строки или ее части;

**substr(…)** – формирование подстроки из строки с заданной позиции до конца или заданной длины.

**Примеры:**

```
if (SS.empty())cout << "Строка пуста.";
S.insert(6,S1,5,10); /* вставка в строку S, начиная с 6 позиции 10 символов из S1, начиная с 5-го */
```

# Класс string. Методы (2)

**`replace(…)`** – замена фрагмента строки на несколько символов, другую строку или фрагмент другой строки;

**`find(…)`** – поиск в строке первого вхождения другой строки, начиная с начала или указанной позиции, возвращает позицию вхождения или string::npos (=-1);

**`rfind(…)`** – поиск в строке последнего вхождения другой строки, начиная с начала или указанной позиции, возвращает позицию вхождения или string::npos;

**`find_first_of(…)`**, **`find_last_of(…)`** – поиск первого или последнего вхождения любого из символов второй строки, возвращает номер символа или string::npos;

**`find_first_not_of(…)`**, **`find_last_not_of(…)`** – поиск первого или последнего вхождения любого из символов, не из второй строки, возвращает номер символа или string::npos;

**`c_str()`** – возвращает указатель типа char * на хранимую С-строку.

**Примеры:**

```
i = S.find(S2,5);  /* номер позиции, начиная с которой подстрока S2
        входит в строку S, поиск ведется с 5-й позиции */
char *str = S.c_str();  // «преобразование» в С-строку
```

# 7.3 Наследование

***Наследование*** – механизм конструирование новых более сложных производных классов из уже имеющихся базовых посредством добавления полей и методов.

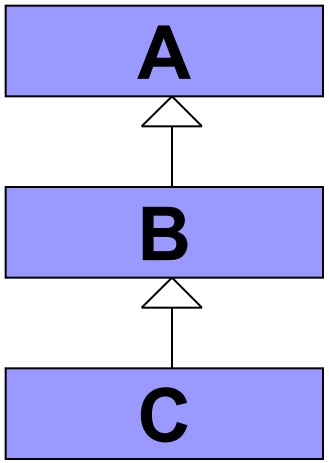
`class` **Имя_производного_класса** `[final]`:

**Вид_наследования Имя_базового_класса{…};**

`final` запрещает дальнейшее наследование

| Вид наследования | Объявление компонента в базовом классе | Видимость компонента в производном классе |
|---|---|---|
| `private` | `private`<br>`protected`<br>`public` | не видимы<br>`private`<br>`private` |
| `protected` | `private`<br>`protected`<br>`public` | не видимы<br>`protected`<br>`protected` |
| `public` | `private`<br>`protected`<br>`public` | не видимы<br>`protected`<br>`public` |

# Конструкторы и деструкторы производных классов




**A() : [Конструкторы_полей] {…}**

**B() :A() [ ,Конструкторы_полей] {…}**

**C() :B() [ ,Конструкторы_полей] {…}**

При объявлении объектов производного класса ***всегда*** вызывается конструктор базового класса, используемый для инициализации наследуемых полей:

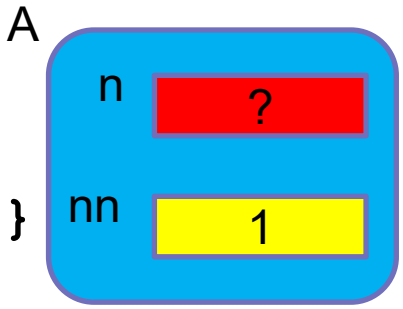
1) если в списке инициализации конструктора производного класса вызов конструктора базового класса есть, то вызывается он;

2) если в списке инициализации конструктора производного класса вызов конструктора базового отсутствует, то *автоматически вызывается конструктор базового класса без параметров!*

# Вызов конструкторов и деструкторов для объектов производных классов

Ex07_05

```
#include <stdio.h>
class TNum {
    public: int n;
        TNum(int an):n(an) {puts("TNum(an)");}
        TNum() {puts("TNum()");}
        ~TNum(){puts("~TNum");}
   };
class TNum2:public TNum {
    public: int nn;
        TNum2(int an):nn(an) {puts("TNum2(an)");}
        ~TNum2(){puts("~TNum2");}
   };
int main() {
   TNum2 A(1);
   return 0;
}
```

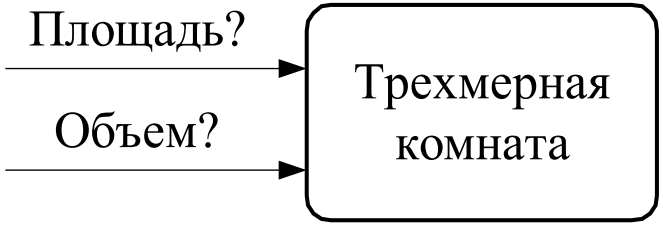


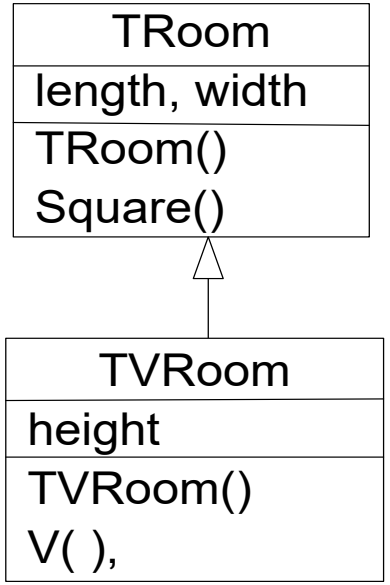

**Неявный вызов конструктора TNum()**

**TNum()**
**TNum2(an)**
**~TNum2**
**~TNum**

Если забыть вызвать конструктор базового класса, то при отсутствии конструктора без параметров получим ошибку компиляции, а при наличии – неинициализированные поля…

# Пример Наследование Ex07_06

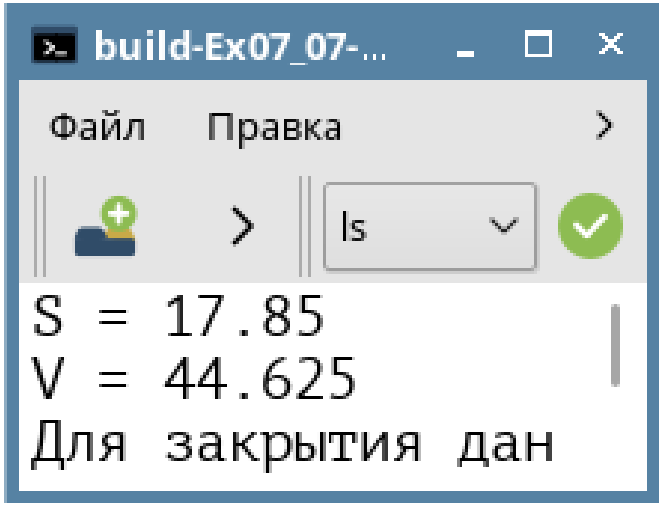



```
#include <iostream>
#include <Room.h>
using namespace std;
class TVRoom:public TRoom{
    private: float height;
    public:
        TVRoom(float l,float w,float h):
                                        TRoom(l,w), height(h){}
        float V(){
            return Square()*height;
        }
};
```

# Пример. Наследование (2)

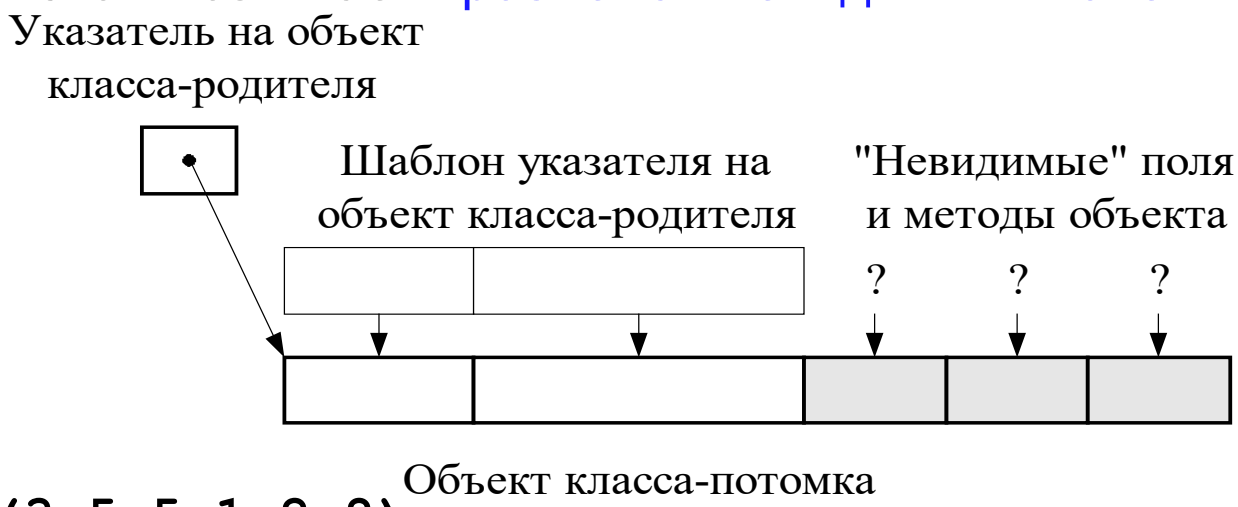
```
int main(){
      TVRoom A(3.5,5.1,2.5);
      cout << "S = " << A.Square() << endl;
      cout << "V = " << A.V() << endl;
}
```



# Присваивание указателей в иерархии

**Допустимо указателю на объект базового класса присваивать адрес объекта производного класса!!!**

Однако при этом возникает проблема «невидимых» полей:




Ex07_06

```
TVRoom E(3.5,5.1,2.8);
TRoom *pC = &E;
s = pC->Square();
v1 = pC->V();   // ошибка!
```

Объект производного класса, но указатель его не видит!

**Правильно:**

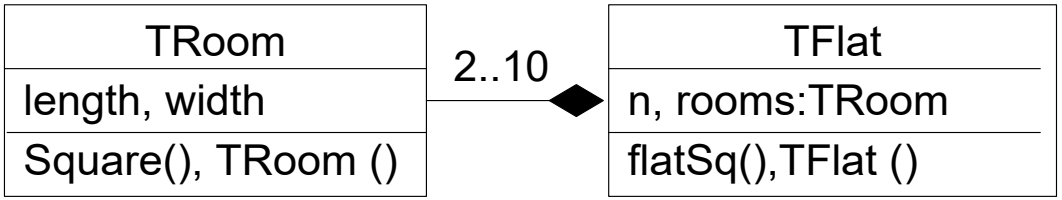
```
v1 = reinterpret_cast<TVRoom *>(pC)->V();
```

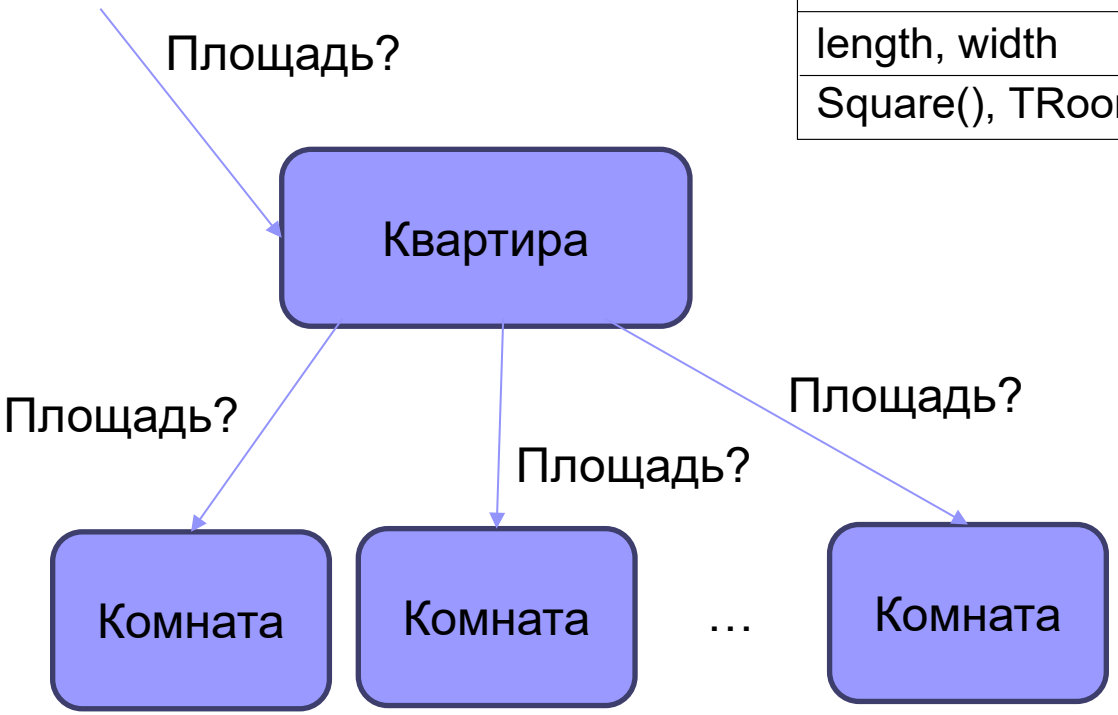
или для полиморфного объекта:

```
v1 = dynamic_cast<TVRoom *>(pC)->V();
```

# 7.4 Композиция

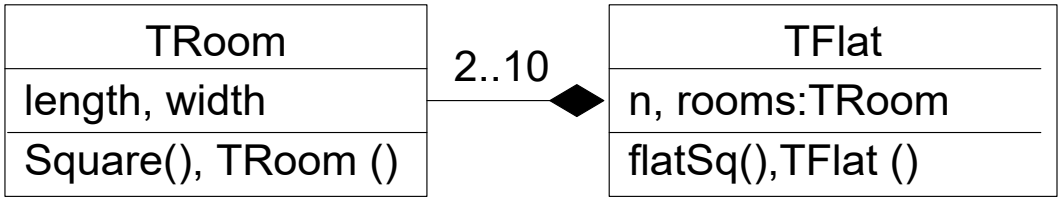
***Композиция*** – включение объектов одного класса в объекты другого. Реализуется механизмом поддержки объектных полей. Ex07_07




Квартира без комнат не имеет бывает, т.е. теряет функциональное назначение!

# Описание класса TRoom




```
#ifndef ROOM_H
#define ROOM_H

class TRoom {
  private: float length, width;
  public:    TRoom(){}
    TRoom(float l, float w){
        length = l;
        width = w;
    }
    float Square(){
            return length* width;
        }
};

#endif // ROOM_H
```

Наличие конструктора без параметров обязательно, так как создается объектное поле Массив объектов!

# Инициализация массива объектов в конструкторе

**А. Инициализация массивом объектов**:

```
TRoom rr[3] = {TRoom(3,5.1),TRoom(2.2,2),TRoom(3,3)};
TFlat A(3,rr);
```

1) *При заданном(неизменяемом) количестве элементов:*

```
TFlat(const TRoom r[]):rooms{r[0],r[1],r[2]}{}// конструктор
```

2) *Если количество элементов заранее неизвестно:*

```
TFlat(int an,TRoom r[]):n(an){          // конструктор
    for (int i = 0;i<=n;i++)
        rooms[i] = r[i];
}
```

**Б. Инициализация массивом значений или структур:**

```
float rr[][2] = {{3,5.1},{2.2,2},{3,3}};
TFlat A(3,rr);
```

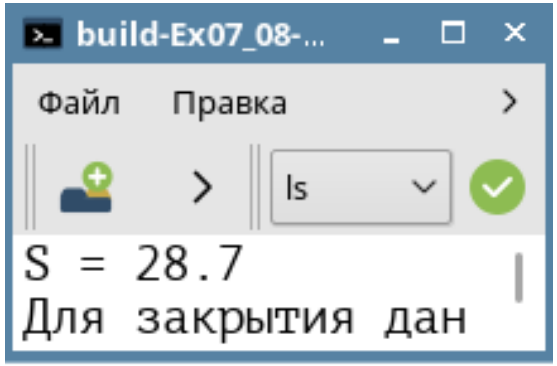
*При заданном(неизменяемом) количестве элементов*

```
TFlat(int an, float r[][2]):n(an), // конструктор
     rooms{{r[0][0],r[0][1]},
           {r[1][0],r[1][1]},
           {r[2][0],r[2][1]}}{}
```

# Пример. Композиция (2)

```
#include <iostream>
#include <Room.h>
using namespace std;
class TFlat {
    private: int n;
        TRoom rooms[10];   // объектное поле
    public:
        TFlat(int an,TRoom r[]):n(an){
            for (int i = 0;i<n;i++)
                rooms[i] = r[i];
        }
        float flatSq(){
            float s = 0;
            for (int i = 0;i<n;i++)s += rooms[i].Square();
            return s;
        }
};
int main() {
    TRoom rr[3] = {TRoom(3,5.1),TRoom(2.2,2),TRoom(3,3)};
    TFlat A(3,rr);
    cout << "S = " << A.flatSq() << endl;
}
```

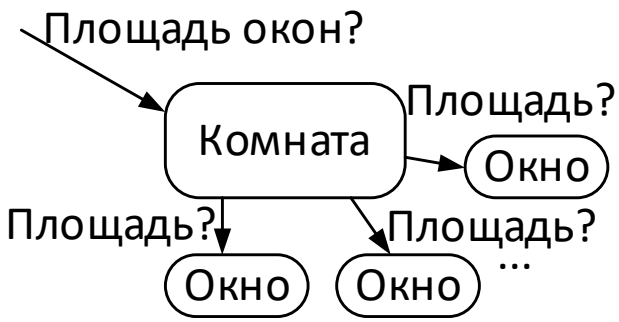
Количество объектов задается при конструировании



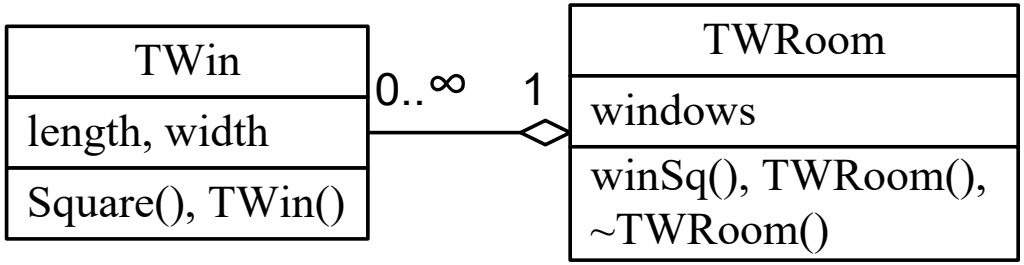
# 7.5 Наполнение (агрегация)

***Наполнение*** – механизм построения классов, при котором объекты строящегося класса могут включать неопределенное количество: от 0 до сравнительно больших значений (на практике обычно до нескольких десятков), объектов других классов.

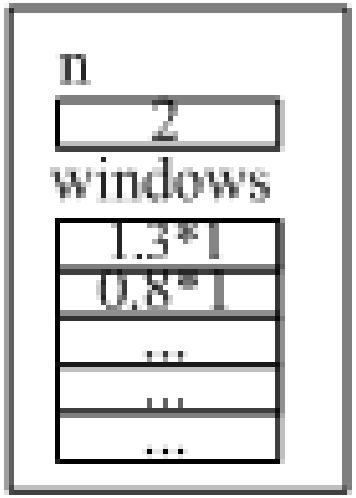
Ex07_08




Без окон комната остается комнатой, т.е. не теряет функционального назначения.




Реализация в виде массива окон




Реализация в виде списка окон

# Пример Наполнение (2)

```
#ifndef WIN_H
#define WIN_H
class TWin {
    private: float length, width;
    public:
        TWin(float l, float w){
            length = l;
            width = w;
        }
        float Square(){
            return length* width;
        }
};
struct SWin {          // тип элемента списка
    TWin win;
    SWin *pSWin;
};
#endif // WIN_H
```

# Пример Наполнение (3)

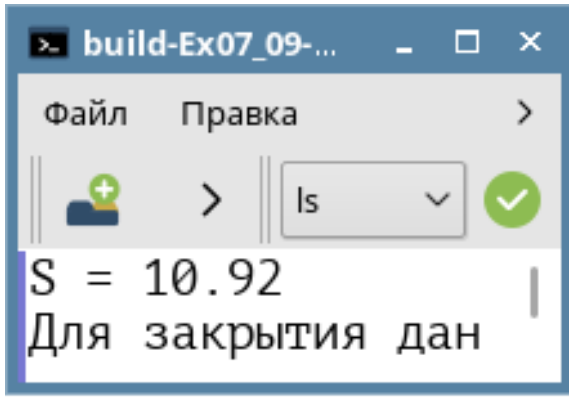
```
#include <iostream>
#include <Win.h>
using namespace std;
class TWRoom {
    private:
        int n;
        SWin *windows;
    public:
        TWRoom(int nn,TWin ww[]):n(nn),windows(nullptr) {
            SWin *q;
            if (n>0) {                // создание списка
                for(int i=0;i<n;i++){
                    q = new SWin{ww[i],windows};
                    windows = q;
                }
            }
        }
```

# Пример Наполнение (4)

```
        float winSq(){                    // вычисление площади
            float s = 0;
            SWin *q = windows;
            for (int i = 0;i<n;i++) {
                s += q->win.Square();
                q = q->pSWin;
            }
            return s;
        }
        ~TWRoom(){                        // деструктор
            SWin *q = windows;
            for (int i=0;i<n;i++) {
                delete q;
                windows = q = windows->pSWin;
            }
        }
};
```

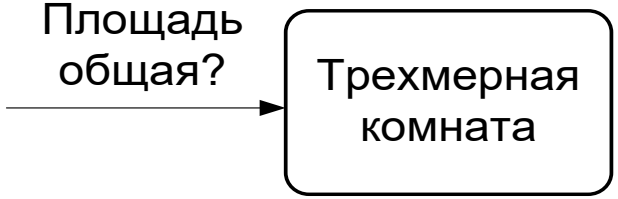
# Пример Наполнение (5)

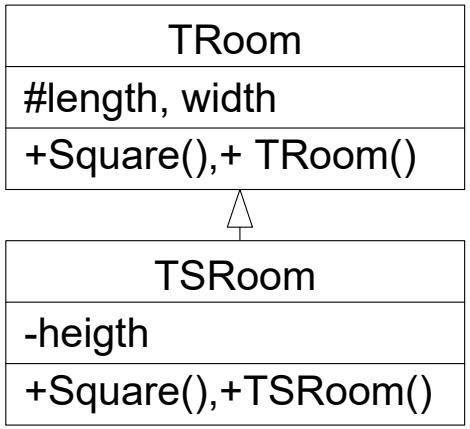
```
int main(){
    TWin rr[] = {TWin(3,2.1),TWin(2.2,2.1)};
    TWRoom A(2,rr);
    cout << "S = " << A.winSq() << endl;
}
```



# 7.6 Простой полиморфизм

**Простой (статический) полиморфизм** – механизм переопределения методов при наследовании, при котором связь метода с объектом выполняется на этапе компиляции (*раннее связывание*). Ex07_09



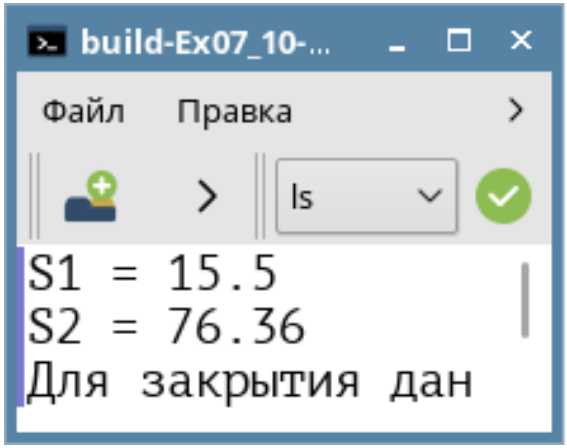




```
#ifndef ROOM_H
#define ROOM_H
class TRoom {
    protected: float length, width;
    public:
        TRoom(float l, float w){
            length = l;
            width = w;
        }
        float Square(){
            return length* width;
        }
};
#endif // ROOM_H
```

Поля должны быть доступны в производном классе

# Пример Простой полиморфизм (2)

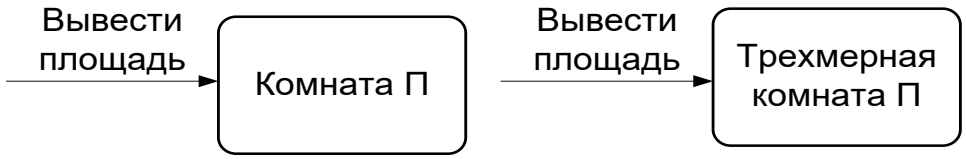
```
#include <iostream>
#include <Room.h>
using namespace std;
class TSRoom:public TRoom{
    private: float height;
    public:
        TSRoom(float l,float w,float h):
                                        TRoom(l,w), height(h){}
        float Square(){
            return 2*(TRoom::Square()+
                                        height*(length+width));
        }
};
int main(){
    TSRoom A(3.1,5,2.8);
    cout << "S1 = " << A.TRoom::Square() << endl <<
            "S2 = " << A.Square() << endl;
}
```

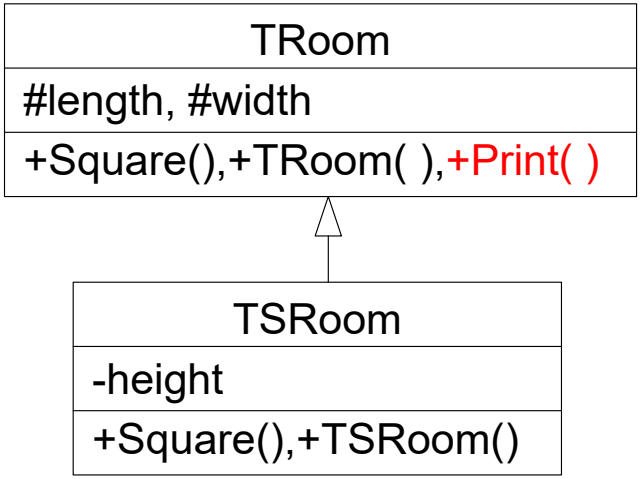



Метод из пространства имен базового класса

# 7.7 Сложный (динамический) полиморфизм

Существует три ситуации, в которых определение *типа* объекта на этапе компиляции программы невозможно, и, следовательно, невозможно правильное подключение переопределенного метода. Ex07_10

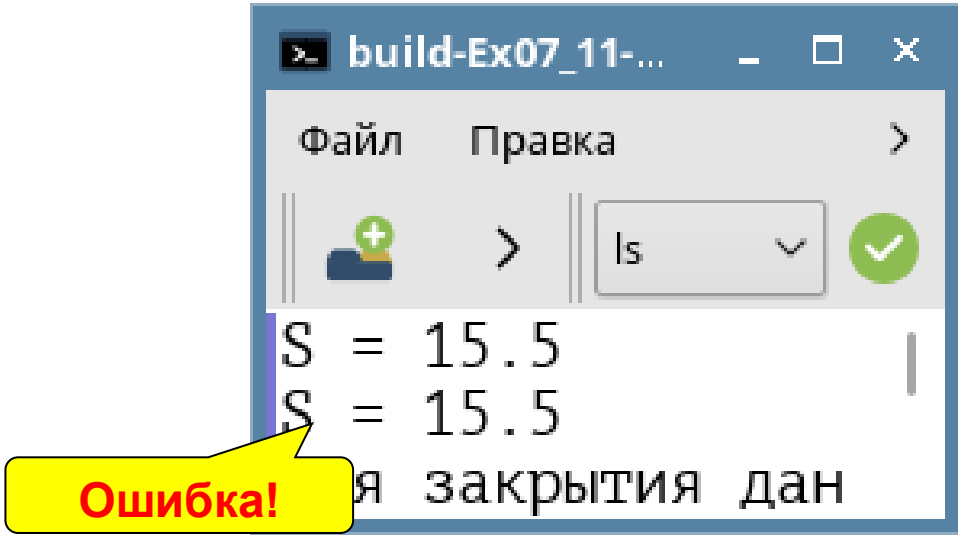


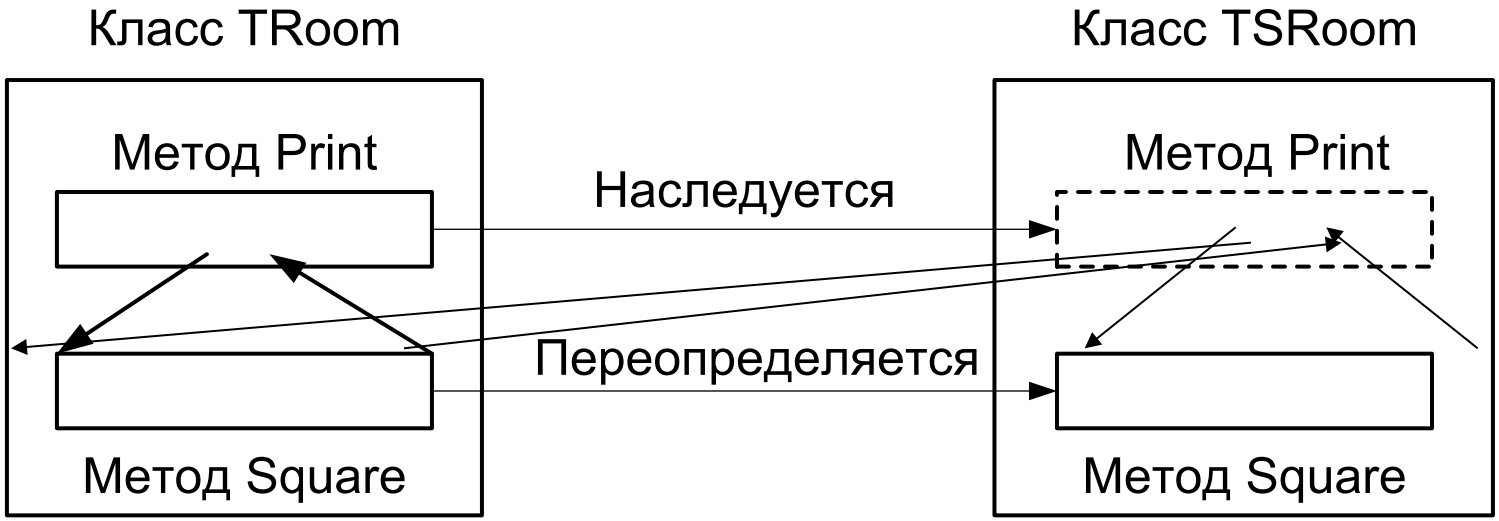

```
#ifndef ROOM_H
#define ROOM_H
#include <iostream>
using namespace std;
class TRoom {
  protected: float length, width;
  public: TRoom(float l, float w){
                length = l; width = w;
            }
        float Square(){
            return length* width;
        }
        void Print(){ cout << "S = " << Square() << endl; }
};
#endif // ROOM_H
```

# Пример Сложный полиморфизм. Ошибка! (2)

```
#include <Room.h>
using namespace std;
class TSRoom:public TRoom{
    private: float height;
    public:
        TSRoom(float l,float w,float h):TRoom(l,w), height(h){}
        float Square(){
            return 2*(TRoom::Square()+height*(length+width));
        }
};
int main(){
    TRoom A(3.1,5);
    A.Print();
    TSRoom B(3.1,5,2.8);
    B.Print();
    return 0;
}
```



# Пояснение к ошибке




При **раннем связывании** нужный аспект полиморфного метода определяется на этапе компиляции программы по типу указателя.

При **позднем связывании** нужный аспект полиморфного метода определяется на этапе выполнения программы по типу объекта, для которого вызывается метод.

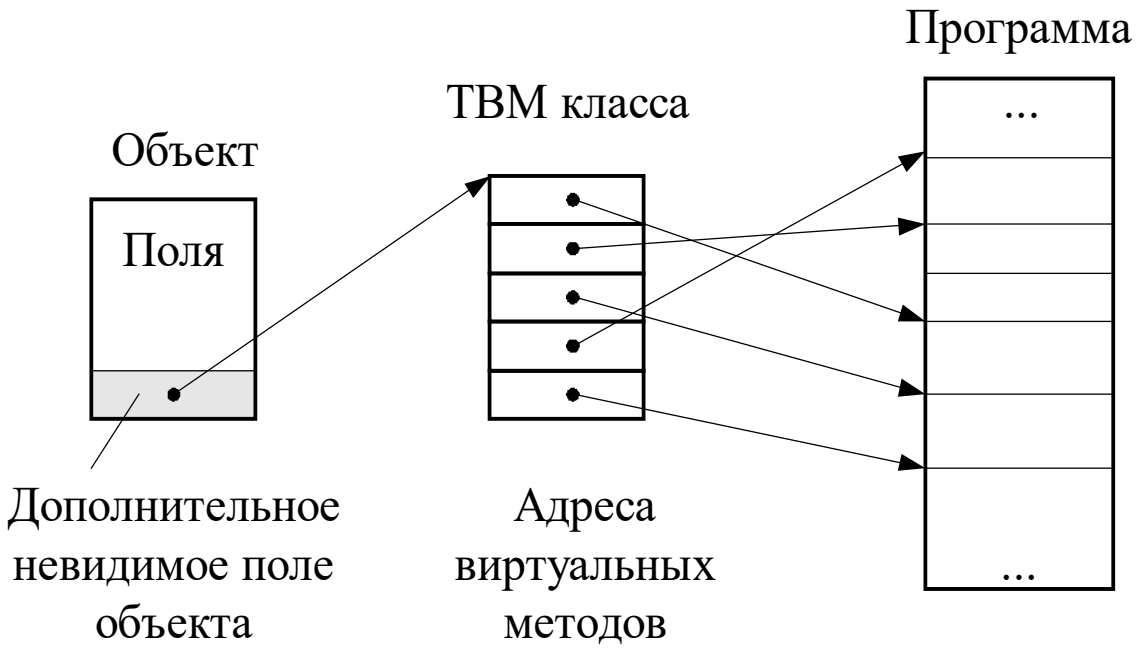
# Реализация динамического полиморфизма

Для организации динамического полиморфизма необходимо:

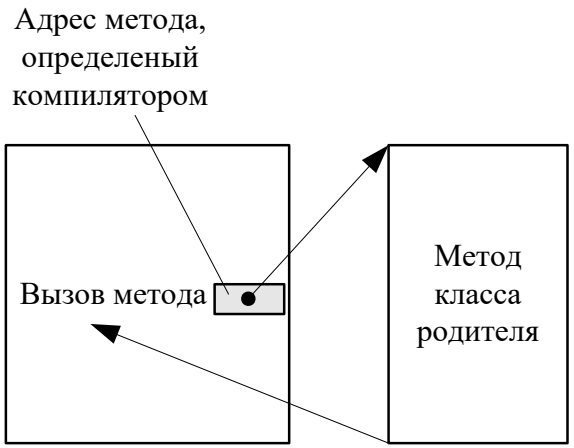
1) переопределяемые методы описать служебным словом ***virtual***;

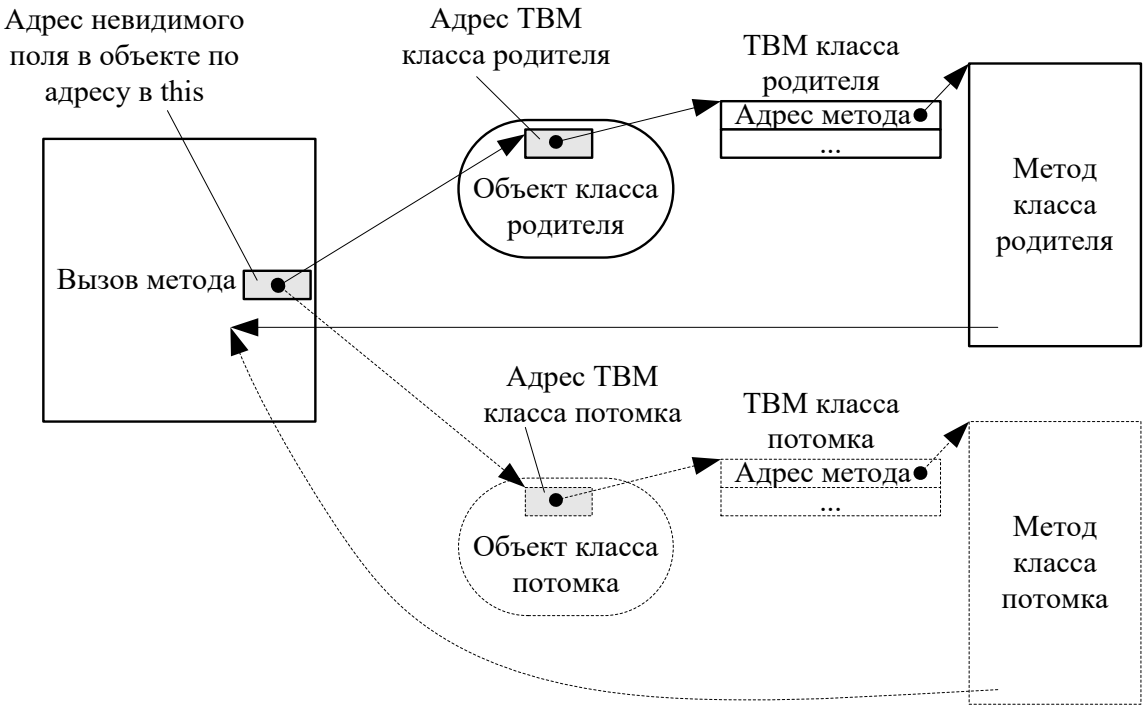
2) вызвать конструктор прежде, чем произойдет первое обращение к виртуальным полиморфным методам.

Подключение осуществляется с использованием ***таблицы виртуальных методов*** (ТВМ), которая создается компилятором при наличии в классе виртуальных методом и адрес которой заносится в невидимое поле объекта при выполнении конструктора.



# Различие раннего и позднего связывания




Раннее связывание – адрес метода определяется на этапе компиляции по *объявленному* типу переменной.

Позднее связывание – адрес метода определяется на этапе выполнения по *фактическому* типу объекта через таблицу виртуальных методов класса, адрес которой хранится в объекте.

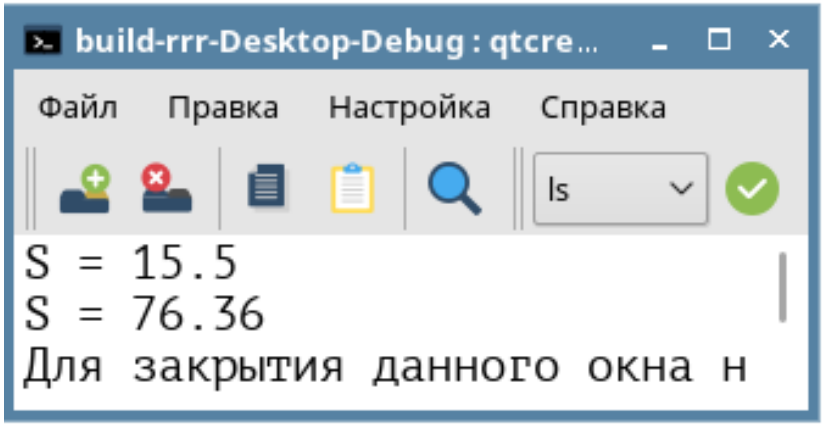
# Исправленный пример

Ex07_10b

```
#ifndef ROOM_H
#define ROOM_H
#include <iostream>
using namespace std;
class TRoom {
    protected: float length, width;
    public: TRoom(float l, float w){
                length = l; width = w;
            }
        virtual float Square(){
            return length* width;
        }
        void Print(){
            cout << "S = " << Square() << endl;
        }
};
#endif // ROOM_H
```

# Исправленный пример (2)

```
#include <Room.h>
using namespace std;
class TSRoom:public TRoom{
    private: float height;
    public:
        TSRoom(float l,float w,float h):
                                    TRoom(l,w), height(h){}
        float Square()override {
            return 2*(TRoom::Square()+
              height*(length+width));
         }
};
int main(){
    TRoom A(3.1,5);
    A.Print();
    TSRoom B(3.1,5,2.8);
    B.Print();
    return 0;
}
```



# 3 случая обязательного использования сложного полиморфизма

**1-й случай** – если наследуемый метод для объекта производного класса вызывает метод, переопределенный в производном классе.

**2-й случай** – если объект производного класса через указатель базового класса обращается к методу, переопределенному производным классом.

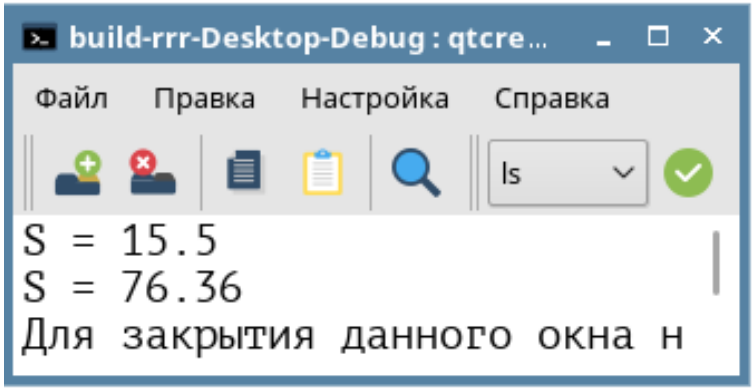
**3-й случай** – если процедура вызывает переопределенный метод для объекта производного класса, переданного в процедуру через *параметр-переменную*, описанный как объект базового класса («процедура с полиморфным объектом»).

# 2-й случай. Обращение через указатель базового класса

Ex07_10c

```
#include <iostream>
#include <Room.h>
class TSRoom:public TRoom{
    private: float height;
    public:
        TSRoom(float l,float w,float h):TRoom(l,w),height(h){}
        float Square() override {
            return 2*(TRoom::Square()+height*(length+width));
        }
};
int main(){
    TRoom *pA = new TRoom(3.1,5), *pB = new TSRoom(3.1,5,2.8);
    cout << "s1 = " << pA->Square() << endl;
    cout << "s2 = " << pB->Square() << endl;
    return 0;
}
```

Вызов переопределенного метода через указатель на объекты базового класса

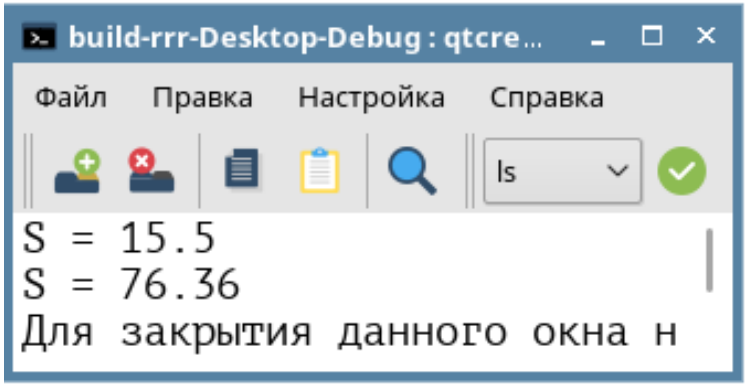


# 3-й случай. Процедура с полиморфным объектом

Ex07_10d

```
#include <iostream>
#include <Room.h>
using namespace std;
class TSRoom:public TRoom{
   private: float height;
   public:
      TSRoom(float l,float w,float h):TRoom(l,w),height(h){}
      float Square() override {
            return 2*(TRoom::Square()+height*(length+width));}
};
void Print(TRoom &r) {
   cout << "S = " << r.Square() << endl;
}
int main(){
   TRoom A(3.1,5);        Print(A);
   TSRoom B(3.1,5,2.8);  Print(B);
   return 0;
}
```

Процедура с полиморфным объектом



# Свойства виртуальных методов класса

1) позднее связывание требует построения ТВМ, а следовательно *больше памяти*;

2) вызов виртуальных полиморфных методов происходит через ТВМ, а следовательно *медленнее*;

3) *список параметров* одноименных виртуальных полиморфных методов *должен совпадать*, а статических полиморфных – не обязательно;

4) статический полиморфный метод не может переопределить виртуальный полиморфный метод.

# Дескрипторы override и final

Дескрипторы используются для предоставления дополнительной информации компилятору.

Совокупность аспектов виртуального метода или просто *виртуальный метод* – множество совпадающих по прототипу (одноименных методов с одинаковыми списками параметров) методов, объявляемых в иерархии.

Дескриптор ***override*** – используется для обозначения совокупности аспектов виртуального метода в иерархии, начиная со второго аспекта.

Дескриптор ***final*** – при использовании в методе обозначает последний аспект семейства виртуальных методов в иерархии.

# Уточняющие описания override и final

Ex07_11

```
class A{
  public:
    virtual void func(){}        // исходные аспекты виртуальных методов
    virtual void f(int){}
    virtual void g()const{}
    virtual ~A(){} };
class B:public A{
  public:
    virtual void f(int) override{}  // переопределяющий метод
    virtual void fumc() override{}
    virtual void f(long) override{}
    virtual void f(int) const override{}
    virtual int f(int) override{}
    virtual void g() const final;  // последний аспект метода
    virtual void g(long);        // новое семейство виртуальных функций
    virtual ~B()override {}
};
```

Использование описателей позволяет компилятору находить ошибки!

Если класс содержит виртуальные методы, то деструктор обязательно виртуальный!!!

# 7.8 Абстрактные методы и классы

Абстрактный метод.

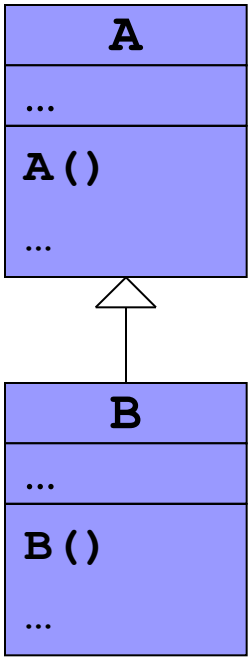
```
class AClass  {
        …
        virtual int Fun(int,int)=0;
    }
```

Класс, содержащий абстрактный метод, называется абстрактным.

Объекты абстрактного класса создавать запрещено !

# 7.9 Приведение типов объекта

Различают приведения:
↑- восходящее;
↓- нисходящее.




В C++ для приведения типов переменных используют:

1) (Тип)Имя_переменной - используется в Си/C++ для любых типов, ничего не проверяет;

2) static_cast <Тип>(Имя_переменной)- используется в C++ для любых типов, проверяет допустимость операции, например не позволит преобразовать целое число в адрес;

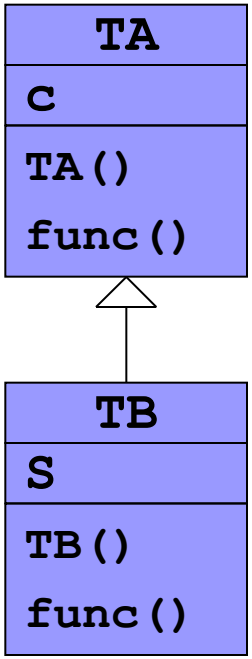
3) const_cast <Тип>(Имя_переменной)- используется для назначения или отмены константности;

4) reinterpret_cast <Тип_указателя> (Указатель_или_интегральный_тип)- используется в C++ для указателей, ничего не проверяет;

5) dynamic_cast <Тип_указателя_на_объект> (Указатель_на_объект) – используется в C++ для полиморфных классов, в VS требует указания опции компилятора /GR (см. меню Project/Properties/Configuration Properties/ C_C++/Language/Enable Run-Time Type Info = Yes), если приведение невозможно, то возвращает nullptr.

# Пример Приведение типов объектов Ex07_12

```
#include <iostream>
using namespace std;
#include <string.h>
class TA  {
  protected:      char c;
  public: TA(char ac):c(ac){}
      virtual void func(){
          cout << c << endl;
      }
};
class TB:public TA  {
      char S[10];
  public: TB(char c,const char *aS):TA(c){
              strcpy(S,aS);
          }
          void func()override {
              cout << c << ' ' << S << endl;
          }
};
```



# Пример. Приведение типов объектов (2)

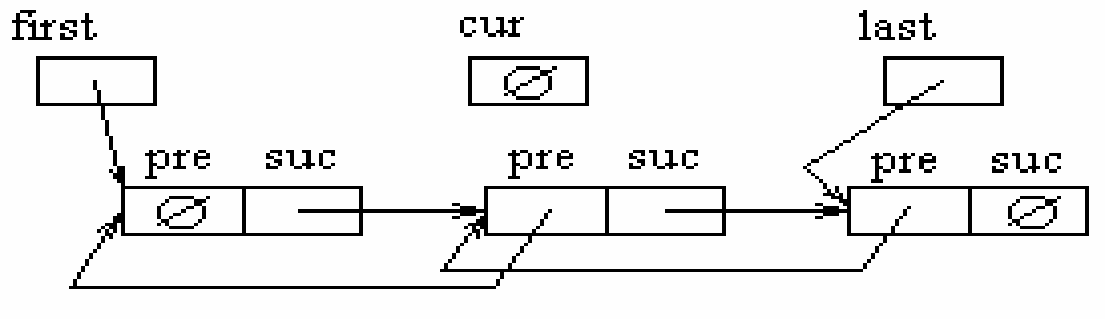
```
int main(int argc, char* argv[])
{ TA *pA=new TA('A'),*pC=new TB('A',"AB");
  TB *pB=new TB('A',"AC");

  ((TA *)pB)->func();
  reinterpret_cast<TA *>(pB)->func();
  static_cast<TA *>(pB)->func();
  dynamic_cast<TA *>(pB)->func();

  ((TB *)pC)->func();
  reinterpret_cast<TB *>(pC)->func();
  static_cast<TB *>(pC)->func();
  dynamic_cast<TB *>(pC)->func();

  ((TB *)pA)->func();
  reinterpret_cast<TB *>(pA)->func();
  static_cast<TB *>(pA)->func();
 // dynamic_cast<TB *>(pA)->func();
  if (TB *pD=dynamic_cast<TB *>(pA)) pD->func();
  else cout<<"Cast Error"<<endl;
  return 0;}
```

Восходящее приведение

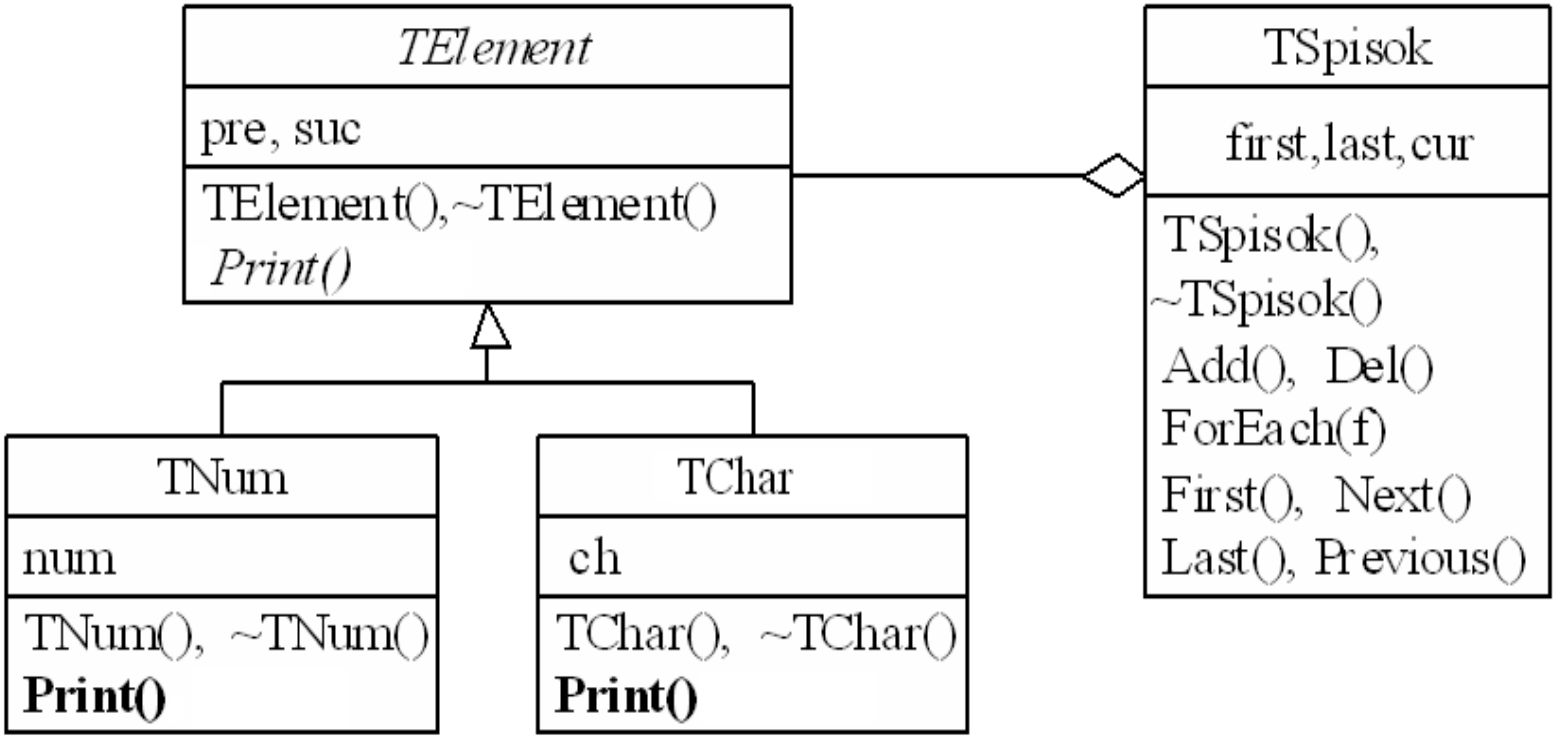
Нисходящее приведение

Ошибка! Приведение не корректно

# 7.10 Контейнер «Двусвязный список» Ex07_13

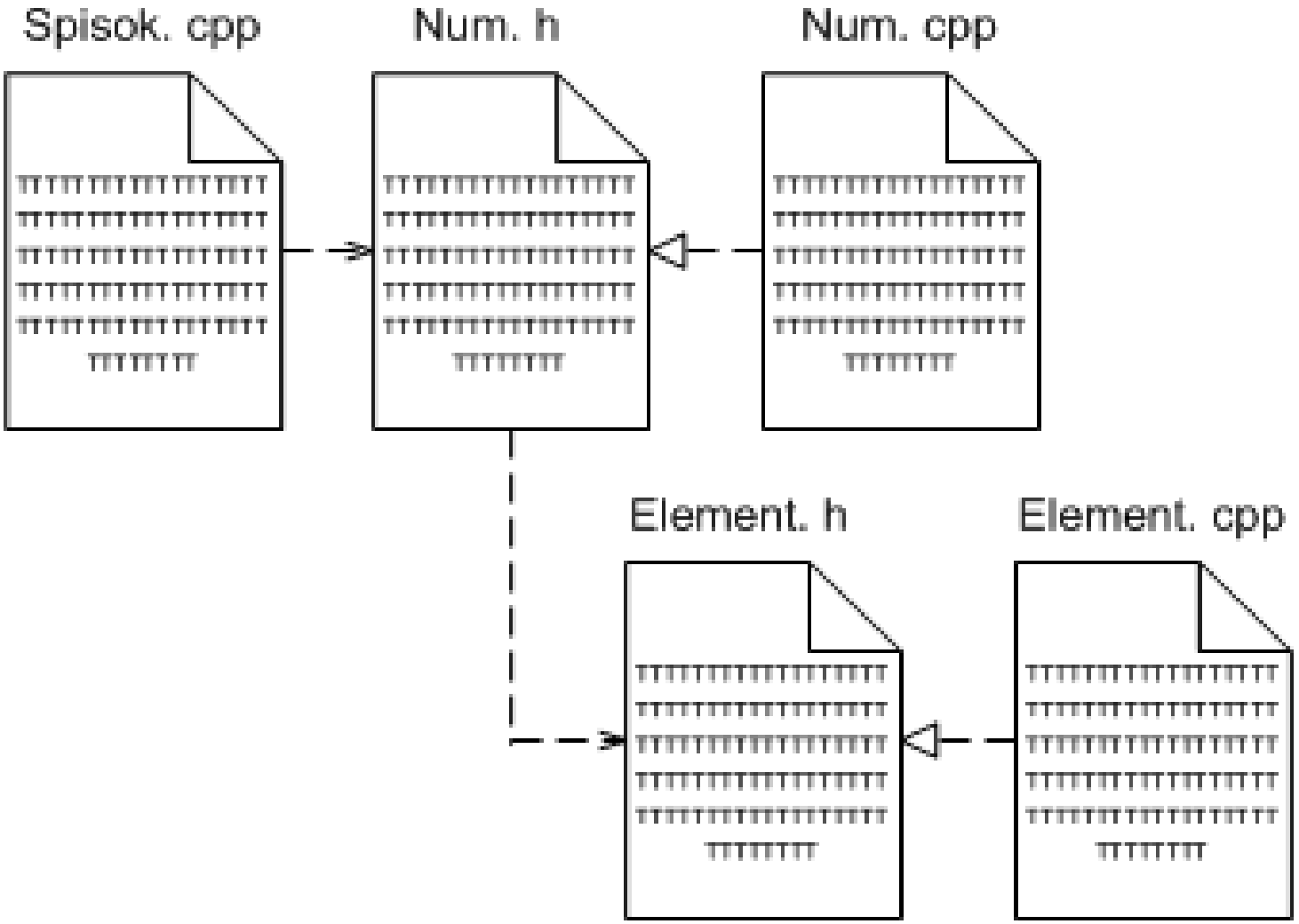



## Диаграмма классов



# Контейнер «Двусвязный список» (2)

## Диаграмма компоновки



# Контейнер Двусвязный список. Файл Element.h

```
#ifndef ELEMENT_H
#define ELEMENT_H
#include <stdio.h>
class TElement
  {  public:    TElement *pre,*suc;
            TElement() { pre = suc = nullptr;}
            virtual ~TElement();
            virtual void Print()=0;
  };
class TSpisok
  {  private:  TElement *first,*last,*cur;
     public:   TSpisok() {first = last = cur = nullptr;}
          ~TSpisok();
          void Add(TElement *e);
          TElement *Del();
          void ForEach(void (*f)(TElement *e));
          TElement *First(){return cur=first;}
          TElement *Next(){return cur=cur->suc;}
          TElement *Last(){return cur=last;}
          TElement *Previous(){return cur=cur->pre;}
  };
#endif // ELEMENT_H
```

# Контейнер Двусвязный список. Файл Element.cpp

```
#include "Element.h"
TElement::~TElement()
{ puts("Delete TElement.");
}
TSpisok::~TSpisok()
{  puts("Delete TSpisok");
   while ((cur=Del())!=nullptr)  { cur->Print();
                                   delete(cur);  }
}
void TSpisok::Add(TElement *e)
{  if (first== nullptr) first=last=e;
   else     { e->suc=first;
              first->pre=e;
              first=e;  }
}
```

# Контейнер Двусвязный список. Файл Element.cpp (2)

```
TElement *TSpisok::Del(void)
{  TElement *temp=last;
   if (last!= nullptr)
            {last=last->pre;
             if (last!=nullptr) last->suc=nullptr;
            }
   if (last==nullptr) first=nullptr;
   return temp;
}
void TSpisok::ForEach(void (*f)(TElement *e))
{  cur=first;
   while (cur!=nullptr)
            {(*f)(cur);
              cur=cur->suc;
            }
}
```

# Контейнер Двусвязный список. Файл Num.h

```
#ifndef NUM_H
#define NUM_H
#include "Element.h"
class TNum:public TElement
{ public: int num;
          TNum(int n):TElement(),num(n) {}
          ~TNum() override;
          void Print() override;
};
class TChar:public TElement
{ public: char ch;
          TChar(char c):TElement(),ch(c) {}
          ~TChar() override;
          void Print() override;
};
void Show(TElement *e);
#endif // NUM_H
```

# Контейнер Двусвязный список. Файл Num.cpp

```
#include "Num.h"

TNum::~TNum() {
   puts("Delete TNum.");
}
void TNum::Print() {
   printf("%d ",num);
}
TChar::~TChar() {
   puts("Delete TChar.");
}
void TChar::Print() {
   printf("%c ",ch);
}
void Show(TElement *e) {
   e->Print();
}
```

# Контейнер Тестирующая программа

```
#include "Num.h"
#include <string.h>
#include <stdlib.h>
int main()
{   TSpisok N;
    char str[10];
    int k,i;
    TElement *p;
    while(printf("Input numbers, strings or <end>:"),
          scanf("%s",str),strcmp(str,"end"))
    { k=atoi(str);
      if (k||(strlen(str)==1 && str[0]=='0'))  p=new TNum(k);
      else p=new TChar(str[0]);
      N.Add(p);
    }
    puts("All list:");
    N.ForEach(Show);
```

# Контейнер Тестирующая программа(2)

```
  p=N.First(); k=0;
  while (p!=nullptr)
   { if (TNum *q=dynamic_cast<TNum *>(p)) k+=q->num;
  // VS установить создание RTTI: /GR в Project\Settings…
     p=N.Next();
   }
  printf("Summa= %d\n",k);
  p=N.Last();
  i=0;
  while (p!= nullptr)
   { if (TChar *q=dynamic_cast<TChar *>(p)) str[i++]=q->ch;
     p=N.Previous();
   }
  str[i]='\0';
  printf("String= %s\n",str);
  return 0;
}
```